HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

Design and Development of a Management and Monitoring System for Extra-Curricular Education Centers in Compliance with Circular 29

BUI HA MY
my.bh225987@sis.hust.edu.vn

Program: Global ICT

Supervisor: M.Sc. Le Tan Hung

Department: Computer Science

School: School of Information and Communications Technology

HANOI, 06/2026

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

Design and Development of a Management and Monitoring System for Extra-Curricular Education Centers in Compliance with Circular 29

BUI HA MY
my.bh225987@sis.hust.edu.vn

Program: Global ICT

Supervisor: M.Sc. Le Tan Hung

Signature

Department: Computer Science

School: School of Information and Communications Technology

HANOI, 06/2026

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to the lecturers of the School of Information and Communication Technology, Hanoi University of Science and Technology. During my years of study, I received invaluable knowledge, guidance and support that built the academic and technical foundation for this graduation thesis.

I am especially grateful to my supervisor, M.Sc. Le Tan Hung, for his dedicated guidance, insightful feedback and continuous encouragement throughout this project. His suggestions and perspectives helped me approach problems more systematically and improve the practical value of the developed system.

I would also like to thank my family for their unconditional love, trust and constant support. Their encouragement gave me the confidence and motivation to overcome challenges and complete this academic journey. My sincere thanks also go to my friends for their companionship, discussions and encouragement during both my studies and the development of this thesis.

Finally, I would like to thank myself for remaining persistent, disciplined and committed throughout this journey. Completing this thesis marks an important milestone in both my academic and personal development.

ABSTRACT

The increasing demand for extra classes has created significant challenges for tutoring centers in managing classes, student registrations, teaching activities, attendance, tuition fees and operational reports. Many centers still rely on paper documents, spreadsheets, or disconnected software tools, resulting in fragmented data, inefficient workflows and difficulties in tracking teaching and learning activities. Existing educational management systems have partially digitalized administrative tasks, but they often focus on general school management and provide limited support for the operational characteristics and regulatory requirements of tutoring centers in Vietnam, particularly the compliance requirements introduced by Circular No. 29/2024/TT-BGDĐT [[1]].

To address these issues, this thesis adopts the approach of developing a centralized web-based management system for tutoring centers. This approach enables users to access the system through web browsers while allowing tutoring centers to manage and maintain operational data in a unified, scalable and easily maintainable environment.

Based on this approach, the thesis designs and implements EduCenter, a web application that supports core workflows of tutoring centers, including class publication, student account registration, enrollment processing, automated session generation, attendance tracking, score management, monthly tuition management, notification delivery and operational reporting. The system is developed using Next.js, React and TypeScript for the frontend and Node.js, Express.js and MySQL for the backend.

The main contributions of this thesis include transforming selected regulations of Circular No. 29/2024/TT-BGDĐT into executable business rules integrated directly into operational workflows, developing an automatic tuition generation mechanism based on actual attendance records and implementing automatic session generation with teaching schedule conflict detection. The final result is a web-based tutoring center management system capable of supporting essential business operations while improving data consistency, operational transparency and compliance with educational regulations.

Student

(Signature and full name)

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Full Expression
API	Application Programming Interface
CRUD	Create, Read, Update and Delete
DBMS	Database Management System
ERD	Entity-Relationship Diagram
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
RBAC	Role-Based Access Control
RDBMS	Relational Database Management System
REST	Representational State Transfer
SQL	Structured Query Language
SSR	Server-Side Rendering
UI	User Interface
UML	Unified Modeling Language
URI	Uniform Resource Identifier
URL	Uniform Resource Locator

CHAPTER 1. INTRODUCTION

1.1 Motivation

This chapter introduces the background and motivation of the graduation project. The chapter first discusses practical issues faced by tutoring centers and the increasing need for transparent and systematic management solutions. It then presents the objectives and scope of the project, proposes the general solution approach and finally summarizes the organization of the remainder of the thesis. In recent years, the dem and for extra classes outside of school has become increasingly popular in Vietnam, especially at general education levels. For many students, tutoring classes are not only places to consolidate classroom knowledge but also supporting environments to practice study skills, prepare for tests or entrance examinations and shape long-term educational orientations. Along with this dem and, the number of tutoring centers, group-based classes and academic enrichment facilities has also risen. Consequently, the operations of a tutoring center are no longer limited to opening classes and collecting tuition fees, but encompass numerous workflows related to learners, teachers, class schedules, class sizes, attendance tracking, academic results, notifications and financial management.

At a small scale, many centers can manage the aforementioned operations using notebooks, spreadsheets, or fragmented messaging tools. While this approach has the advantages of being simple and easy to start with, it rapidly reveals limitations as the number of classes, students and teachers grows. Registration information may reside in one place, schedules are stored elsewhere, tuition fees are compiled manually and notifications are sent across multiple separate channels. When data becomes scattered, retrieving registration histories, checking tuition payment statuses, reconciling attendance, or aggregating academic performance becomes time-consuming and prone to errors. For administrators, this reduces the ability to grasp the operational status of the center. For teachers, inconsistent tracking of classes and students can also affect teaching quality.

This issue directly impacts students and parents, who should be placed at the center of educational activities. When class information is not publicly and clearly disclosed, parents find it difficult to assess whether a class matches the needs, schedule and conditions of the student. When schedules, attendance, test scores and tuition fees are not updated transparently, it becomes challenging for students and parents to continuously monitor the learning process. In an educational environment, a lack of clarity regarding information not only causes management

inconveniences but can also undermine the trust of learners in the center. A well-operating tutoring center must ensure that administrative decisions not only serve internal efficiency but also contribute to protecting the legitimate rights of students.

In addition to operational requirements, extra tutoring activities in Vietnam are currently subject to increasingly stringent legal regulations. Circular 29/2024/TT-BGDĐT sets forth many crucial requirements for tutoring and extra learning activities, emphasizing the principle of voluntary participation by learners, information transparency in organizing tutoring, financial management accountability and cases where tutoring is prohibited. Notably, teachers currently employed in schools are not allowed to provide paid extra tutoring outside of school to students whom they are officially assigned to teach in the formal curriculum. This regulation adds complexity to the management of tutoring centers, as centers must not only manage classes based on business needs but also control the relationships among teachers, students, schools, grade levels and formal classes.

If the above requirements are not managed systematically, centers may encounter multiple operational risks. A class could be opened without sufficient public disclosure; a student might register for an unsuitable class; a teacher could inadvertently be assigned to provide extra tutoring to a student from their formal classroom; or tuition records, attendance and notifications might lack a verifiable basis when reconciliation is needed. These risks not only impair the administrative efficiency of the center but also jeopardize transparency, fairness and the rights of learners.

Therefore, the problem to be solved is the reorganization of tutoring center management activities towards a centralized, transparent and more controllable approach. If this problem is resolved, centers can minimize operational errors, administrators will have a foundation to monitor class and tuition statuses, teachers will find it more convenient to track the learning process and students and parents will gain access to clearer information. The significance of this problem extends beyond any specific center, as it can be scaled to various educational institutions outside of school that need to manage classes, learners, teachers and compliance obligations. This serves as the practical foundation for selecting the topic of tutoring center management for this graduation thesis.

1.2 Objectives and scope of the graduation thesis

Currently, management activities at tutoring centers are often performed using various groups of tools. Some centers use spreadsheets to store student lists,

class schedules, tuition fees and attendance statuses. Other centers combine online forms, messaging applications and simple accounting software to receive registrations, notify class schedules, or track collections. These tools are easy to deploy and suitable for a small scale; however, when the number of classes, teachers and students increases, data easily becomes scattered, duplicated and difficult to control in terms of modification histories.

In addition, some educational management systems have supported operations such as managing classes, student profiles, schedules, tuition fees and reports. These systems help reduce manual tasks and support administrators in monitoring activities more effectively. Nevertheless, many systems are built for general educational management problems and have not yet focused on the specific characteristics of tutoring centers in Vietnam. These distinct characteristics do not only lie in opening classes, scheduling and collecting tuition fees, but also relate to the relationships among teachers, students, formal schools, grade levels and the legal requirements governing extra tutoring activities.

From a user perspective, a tutoring center management system must simultaneously serve multiple target groups. Administrators need to monitor classes, enrollment sizes, responsible teachers, tuition fees and pending registrations. Center staff require tools to approve registrations and support students. Teachers need to view their assigned classes, take attendance, enter test scores and send notifications related to the learning process. Students need to view open classes, register for courses, track registration statuses, attendance, scores, tuition fees and announcements from the center. If these needs are not accommodated within the same system, the operational process will lack seamlessness and make it difficult to guarantee the rights of learners.

On that basis, the objective of the thesis is to design and build the EduCenter web system to support the management of the core workflows of tutoring centers. The system aims to centralize data regarding classes, teachers, students, registrations, class sessions, attendance tracking, test scores, tuition fees and notifications. Data centralization helps the center reduce manual operations, limit errors in workflow processing and enhance information retrieval capabilities. For students, the system makes the process of accessing classes, registering for courses and tracking academic information clearer.

Another crucial objective of the thesis is to incorporate compliance requirements from Circular 29/2024/TT-BGDĐT into the management process. The system aims to support the control of several vital constraints, such as publicly dis-

closing class information prior to enrollment, managing class sizes, limiting the number of class sessions, tracking relationships between teachers and students in formal schools and maintaining records of registrations, attendance, tuition fees and notifications. Integrating compliance factors into the system helps the center operate more transparently while supporting the reduction of risks arising from assignment decisions or registration requests that have not been adequately verified.

Within the scope of the thesis, EduCenter focuses on core functional groups including publicizing the list of open classes, registering student accounts, registering for classes, approving or rejecting registrations, managing classes and schedules, automatically generating class sessions, tracking attendance by session, entering test scores, managing monthly tuition fees, confirming payments, sending internal notifications and viewing operational reports. The main roles in the system consist of administrators, center staff, teachers and students. The parental role is not separated into an independent account; parent or guardian information is stored within the student profile to facilitate contact when necessary.

The thesis does not aim to build a comprehensive online education platform. The system does not include real-time video classrooms, does not integrate online payment gateways, does not send real emails, SMS, or push notifications, does not support native mobile applications and does not handle multi-center problem contexts. Legal matters outside the scope of the software, such as the business registration of the tutoring facility or the reporting duties of teachers to their formal schools, are also not automated. The focus of the topic is to construct a web system that can operate the basic workflows of a tutoring center following a consistent, transparent and traceable process.

1.3 Tentative solution

Based on the defined objectives and scope, the thesis chooses the approach of building a centralized management web system for tutoring centers. This approach aligns with the operational characteristics of centers because users can access the system through web browsers without installing dedicated software. Concurrently, operational data is stored centrally, which reduces information dispersion among multiple manual tools such as spreadsheets, online forms, or messaging applications.

Regarding technological orientation, the system is constructed following a client-server model with a frontend serving the user interface, a backend processing business logic and a relational database storing information. The interface is developed as a modern web application to support various user groups, including adminis-

trators, center staff, teachers and students. The backend provides APIs for core operations while performing authorization checks, input data validation and rule processing related to course registrations, classes, class sessions, attendance tracking, tuition fees and notifications.

The solution of the thesis is the EduCenter system, a web application that supports managing the core workflows of tutoring centers under a unified process. Students can view the list of open classes, register for accounts, enroll in classes and track their academic information. Center staff can process pending registrations, monitor assigned classes and support students during their learning process. Teachers can view their assigned classes, take attendance, enter test scores and send notifications related to class sessions. Administrators can manage users, teachers, students, classes, tuition fees and operational reports.

A critical orientation of the solution is integrating compliance requirements from Circular 29/2024/TT-BGDĐT directly into the operational workflows of the system. Instead of merely storing class and tuition data, the system aims to support the control of several vital constraints, such as publicly disclosing class information, limiting the number of class sessions, managing class sizes and verifying relationships between teachers and students in formal schools. This approach helps the center operate more transparently and reduces risks arising from registration or assignment decisions that lack verification.

The main contribution of this thesis is the design and implementation of EduCenter, a web-based tutoring center management system that integrates fragmented operational workflows into a unified management process. In addition to supporting core functions such as class registration, teaching activity monitoring, tuition management and notifications, the system embeds selected regulatory constraints derived from Circular No. 29/2024/TT-BGDĐT into enrollment and class management processes. The resulting prototype provides transparent information access, improves traceability of operational activities and establishes a foundation for future extensions of the platform.

This chapter has presented the practical motivation and objectives of developing EduCenter, a web-based tutoring center management system. The chapter has also defined the project scope and proposed a centralized web-based solution that incorporates selected regulatory requirements derived from Circular No. 29/2024/TT-BGDĐT. Based on these foundations, the next chapter investigates the functional requirements and analyzes the major business processes that the system must support.

1.4 Thesis organization

1.4 Thesis organization

The remainder of this thesis is organized as follows.

Chapter 2 presents the requirement survey and analysis for the EduCenter system. The chapter begins by reviewing existing tutoring center management solutions and identifying their limitations. Based on this analysis, it defines the functional and non-functional requirements of the proposed system through general and detailed use case diagrams, business process models and detailed specifications of the principal use cases that guide the subsequent system design.

Chapter 3 introduces the theoretical background and enabling technologies adopted in the development of the system. It discusses the frontend and backend technologies, including TypeScript, React, Next.js, Tailwind CSS, Zustand, Node.js, Express.js, RESTful API architecture, JSON Web Token authentication, role-based access control and the MySQL relational database management system. The chapter also explains the rationale for selecting these technologies to satisfy the project requirements.

Chapter 4 presents the design, implementation and evaluation of the EduCenter system. The chapter describes the overall software architecture, package organization, user interface design, layer design and database design before presenting the completed implementation, the development tools, the implementation results, representative user interfaces, functional testing and the proposed deployment model of the application.

Chapter 5 presents the principal technical contributions of the thesis. It explains how selected educational regulations derived from Circular No. 29/2024/TT-BGDĐT are transformed into executable business rules within the enrollment process, describes the automatic monthly tuition generation mechanism based on attendance records, and introduces the automatic class session generation and teaching schedule conflict detection mechanisms that improve operational efficiency and consistency.

Chapter 6 concludes the thesis by summarizing the achieved objectives, evaluating the overall outcomes of the developed system and discussing its current limitations. The chapter also proposes several directions for future development to further enhance the functionality, scalability and practical applicability of the EduCenter platform.

CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS

This chapter presents the requirement survey and analysis process for the EduCenter system. It first analyzes the current management practices of tutoring centers and the limitations of existing management approaches. Based on this analysis, the chapter identifies the main system functions through a general use case diagram, detailed use case diagrams and several important business processes. The chapter then provides detailed specifications for selected key use cases and discusses the non-functional requirements that should be considered during system design.

2.1 Status survey

In recent years, the dem and for after-school tutoring has continuously risen to meet the needs of knowledge supplementation, exam preparation and the development of student learning capabilities. Along with the expansion of tutoring activities, the number of educational centers and cultural enrichment centers has grown significantly. These centers must simultaneously manage a diverse range of information such as students, teachers, classes, schedules, tuition fees, attendance and other operational activities. As the scale of these centers expands, manual management methods like paper logbooks or spreadsheets no longer meet practical requirements. Data is often scattered, difficult to track, time-consuming to aggregate and highly prone to errors during operations.

Practical surveys indicate that most tutoring centers currently have a dem and for software to support centralized data management and automate business processes. Managers wish to possess the capability to track class status, student enrollment numbers, tuition revenue and teaching schedules in real-time. Teachers require tools that assist in managing student rosters, tracking teaching schedules and conducting attendance more conveniently. For students, actively looking up class information, registering for classes and tracking tuition fees enhances their experience using the services of the center.

Currently, numerous training center management software solutions have emerged in the Vietnamese market, such as Mona eLMS [[2]], Faceworks Education [[3]] and EasyEdu [[4]]. These systems provide a relatively comprehensive set of basic functionalities, including student management, class management, schedule management, tuition management and user access control. Some systems also support statistical reporting and integrate communication tools with students. The adoption of this software significantly reduces the manual workload, increases accuracy in

data management and enhances the operational efficiency of centers.

Although these existing systems meet many common business requirements well, they still present certain limitations. The majority of solutions focus on generic management functions without paying sufficient attention to the specific characteristics of tutoring activities in the context of newly issued legal regulations. Processes such as verifying the validity of classes by grade level and subject, limiting the number of lesson sessions according to regulations, controlling the relationship between teachers and students, or automatically detecting violations of tutoring regulations are still primarily performed manually. Furthermore, automatic schedule generation, managing class registration tied to complex business rules and controlling access permissions according to individual user roles are not yet fully supported in many systems.

Based on the results of the current state survey and the actual needs of users, it is evident that building a tutoring center management system does not stop at merely digitizing data and supporting basic management operations. It also requires the integration of automation mechanisms and compliance checks for legal regulations directly within the operating process. Originating from this practical context, this thesis aims to design and build the EduCenter system, a web-based tutoring center management system that supports centralized management of center entities and business operations while integrating mechanisms to check constraints from Circular No. 29/2024/TT-BGDĐT, thereby improving management efficiency and minimizing errors during operations.

2.2 Functional overview

Before modelling the system formally, it helps to see the whole surface of what it does. This section first lays out the complete feature map, then draws the actors and their use cases and finally walks the business process that ties the features into one working flow. The detailed specification of selected use cases follows in Section 2.3and the supporting ones are collected in Appendix **??**.

2.2.1 General use case diagram

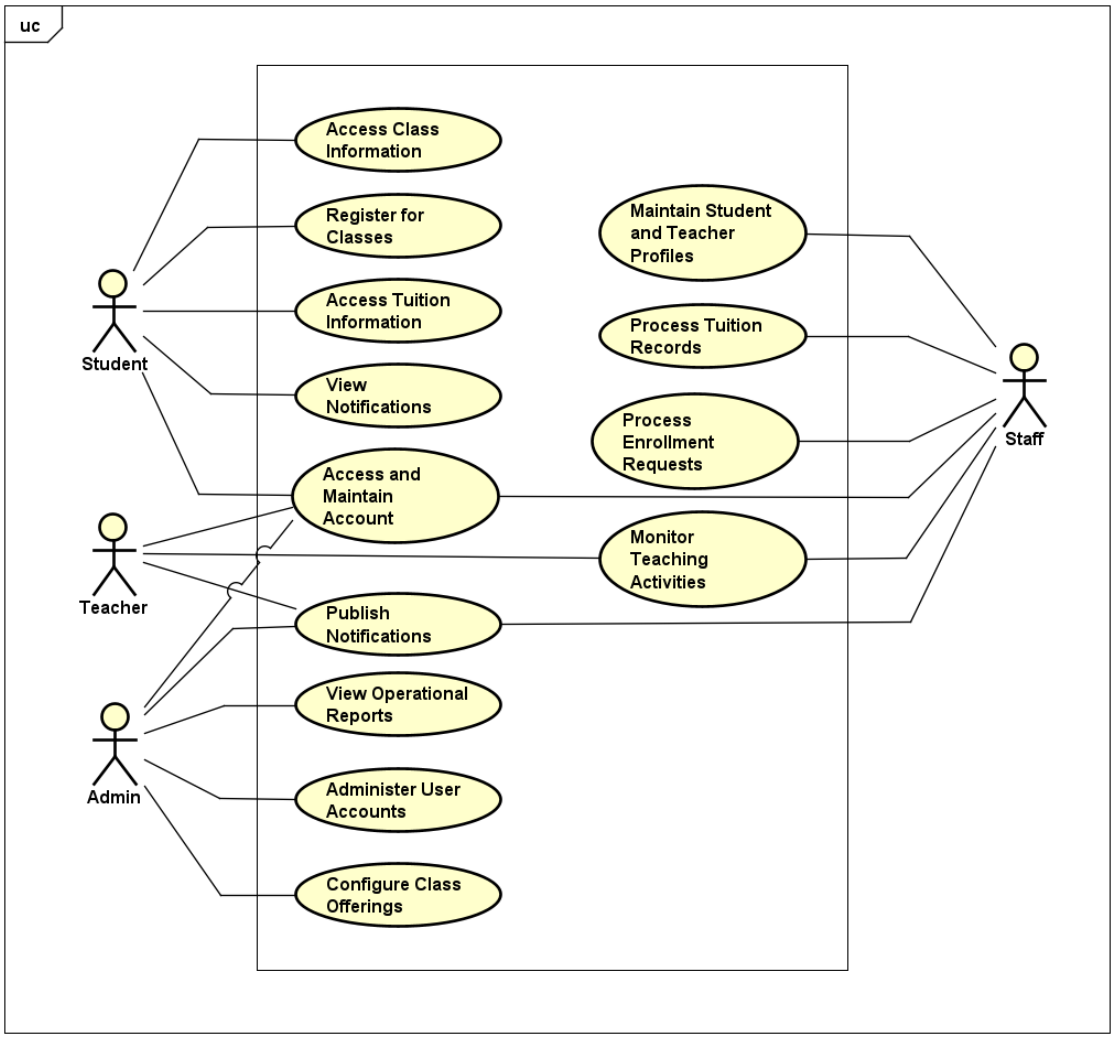


Figure 2.1: General use case diagram

Figure 2.1 presents the general use case diagram of the EduCenter tutoring center management system. The system involves four main actors interacting with it, including Student, Teacher, Staff and Administrator.

The Student is the participant in learning activities at the center and represents the primary user group of the system. Students use the system to manage personal accounts, look up and register for classes, track tuition information and receive notifications related to their learning process.

The Teacher is the individual directly involved in teaching activities at the center. Teachers use the system to manage personal accounts, monitor teaching activities, take attendance, record student learning results and send notifications related to their assigned classes.

The Staff is the entity supporting the daily operations of the center. Staff members use the system to manage personal accounts, maintain student and teacher

profiles, process enrollment requests, manage tuition records, support teaching activities and send notifications when necessary.

The Administrator possesses the highest level of authorization within the system. Administrators use the system to manage personal accounts, administer user accounts and permissions, configure class offerings, send notifications and view operational reports to serve center management and administration.

Parents are an important stakeholder but are not modeled as an actor in the use case diagram because they do not hold an account and do not directly interact with the system. Parent contact information is stored via the guardian phone number in the student profile. In the current version, notifications and reminders for parents are simulated as system notifications and displayed to the students.

Each actor interacts with the system through functions corresponding to their roles and responsibilities, thereby contributing to ensuring that the management, teaching and operations of the tutoring center are executed in an efficient, transparent and convenient manner.

2.2.2 Detailed use case diagram

a, Detailed use case diagram for Register for Classes

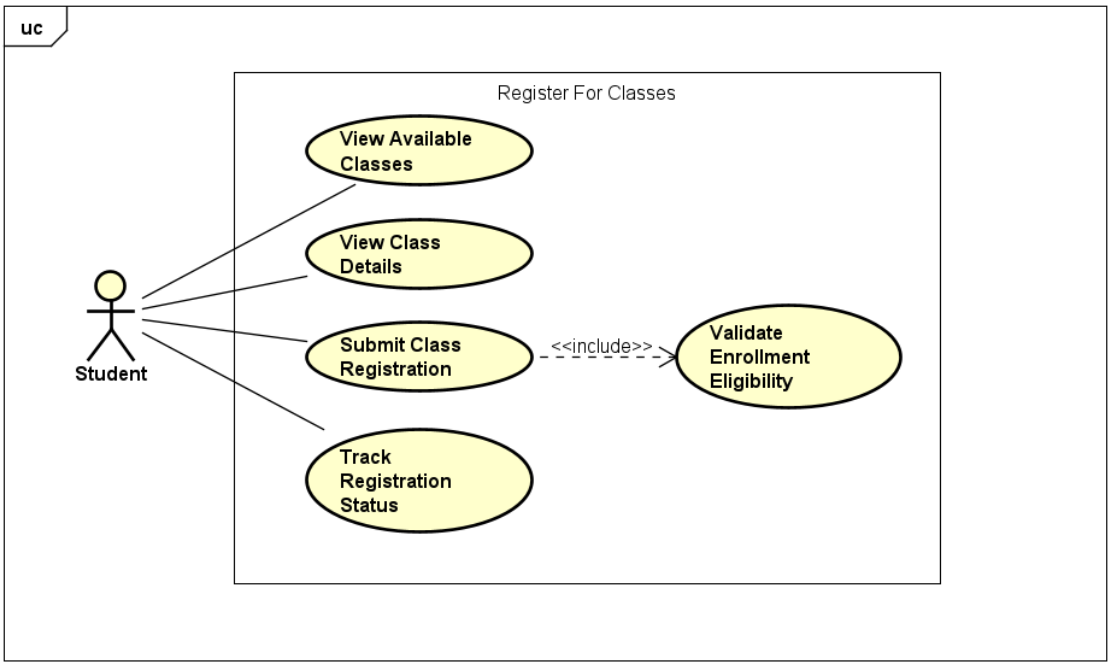


Figure 2.2: Detailed use case diagram for Register for Classes

Figure 2.2 describes the decomposition diagram of the Register for Classes use case. The actor directly executing this use case is the Student. The system supports the Student in activities including viewing the list of available classes, viewing detailed class information, submitting enrollment requests and tracking the reg-

istration status after submission. When a Student performs class registration, the system automatically verifies validity conditions such as class status, current student enrollment numbers, duplicate registration status and constraints related to Circular 29 before recording the registration request.

b, Detailed use case diagram for Configure Class Offerings

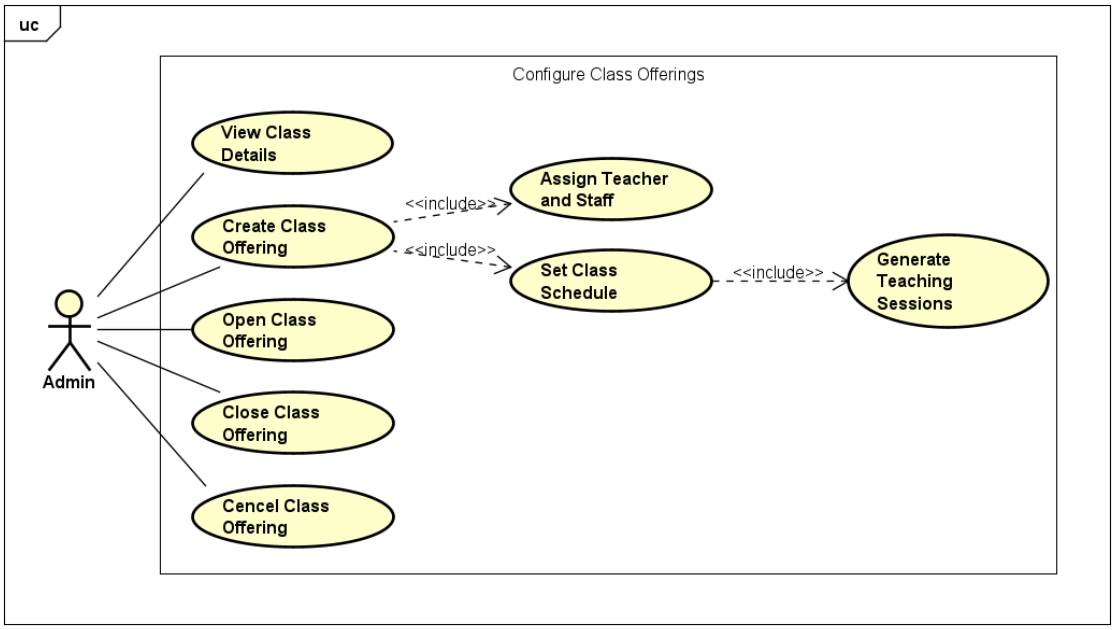


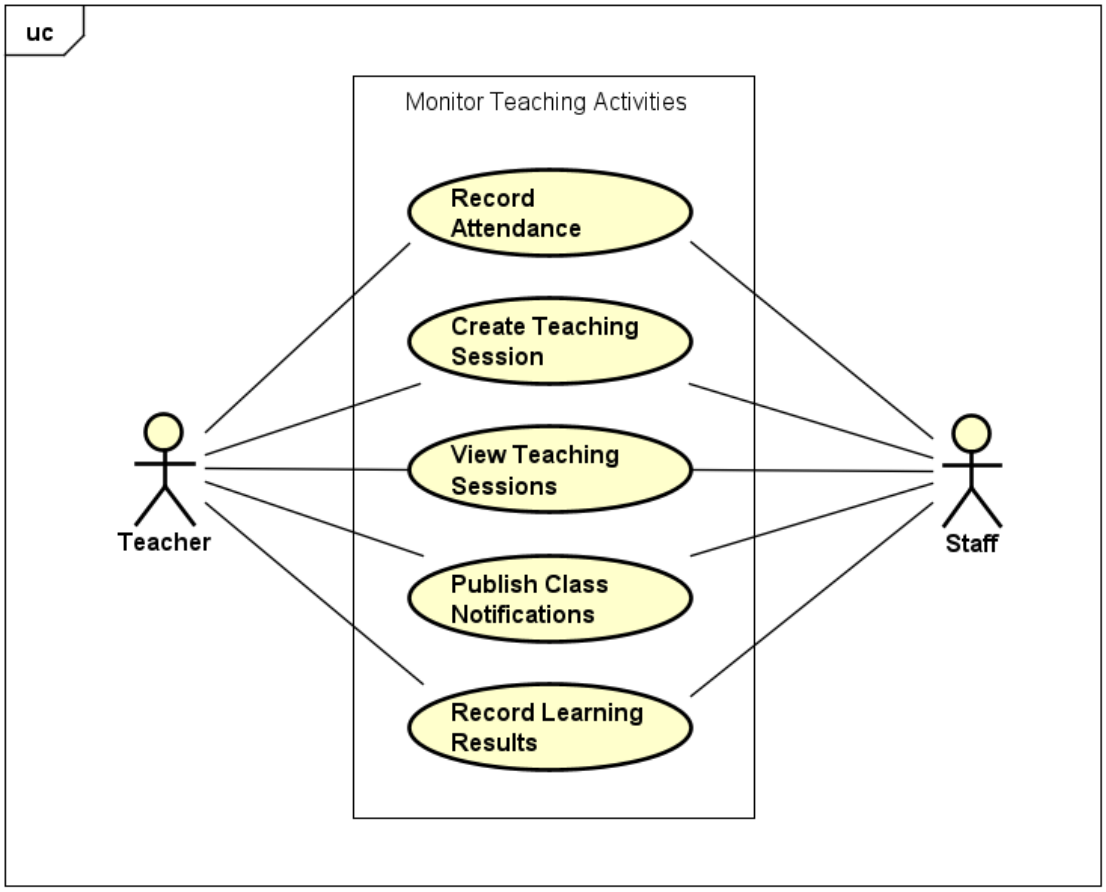
Figure 2.3: Detailed use case diagram for Configure Class Offerings

Figure 2.3 describes the decomposition diagram of the Configure Class Offerings use case. The actor directly executing this use case is the Administrator. The system supports the Administrator in activities including viewing class details, creating new classes, opening classes, closing classes and canceling classes when necessary. During the class creation process, the Administrator needs to assign teachers and staff in charge as well as configure the class schedule. From the configured class schedule, the system automatically generates corresponding class sessions to support the class organization and operational process.

c, Detailed use case diagram for Monitor Teaching Activities

Figure 2.4 describes the decomposition diagram of the Monitor Teaching Activities use case. The actors directly executing this use case are the Teacher and the Staff. The system supports users in activities including viewing the list of class sessions, creating class sessions when necessary, taking student attendance, recording learning results and sending notifications related to the class. In the event that the teacher cannot directly perform these operations, the center staff can assist in updating class session information, attendance, grades and notifications to ensure that teaching activities are maintained continuously.

Figure 2.4: Detailed use case diagram for Monitor Teaching Activities



2.2.3 Business process

a, Business process of Student Registration and Class Enrollment

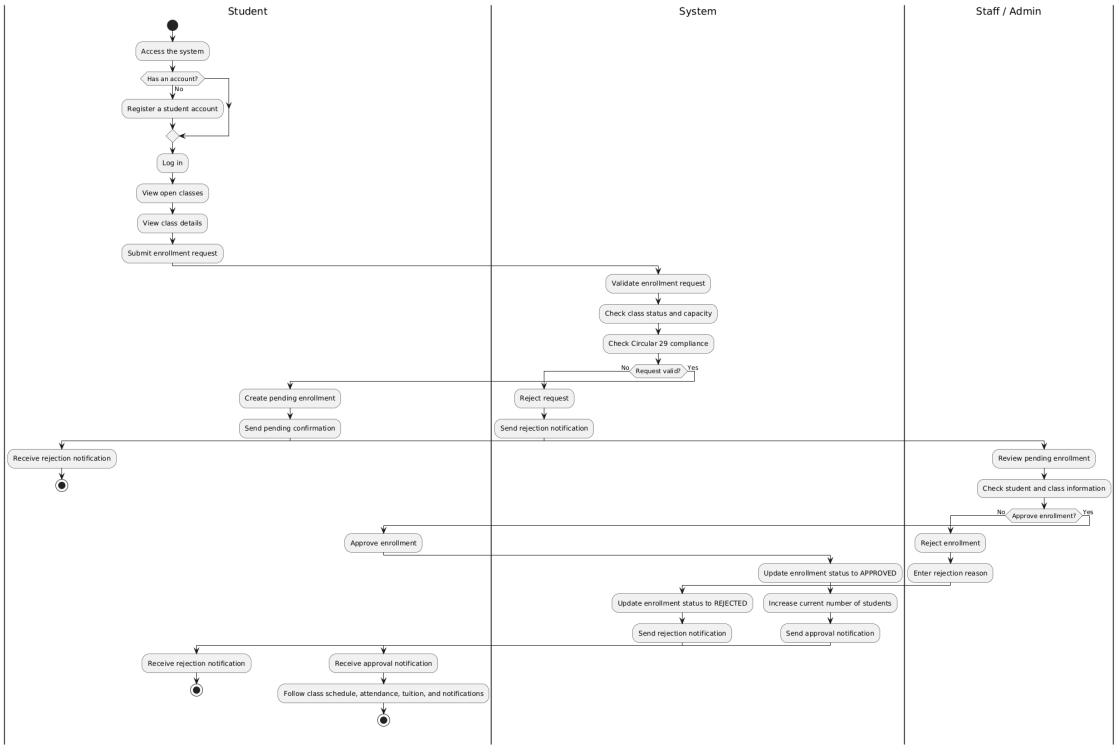


Figure 2.5: Business process of Student Registration and Class Enrollment

Figure 2.5 describes the business process of student registration and class enrollment at the tutoring center. The process begins when a student accesses the system and registers a student account if they do not already have one. After logging in successfully, the student can view the list of available classes and examine detailed information such as the subject, teacher in charge, class schedule, tuition fees and the number of remaining vacancies. After selecting a suitable class, the student submits an enrollment request. The system then validates the request by checking the class status, the current number of enrolled students, duplicate registration conditions and business constraints derived from Circular No. 29/2024/TT-BGDĐT. If the request is valid, the system creates an enrollment record with a pending status and sends a confirmation notification to the student. Next, a staff member or the administrator reviews the pending enrollment request, verifies the student and class information and decides whether to approve or reject the request. If the request is approved, the enrollment status is updated to APPROVED and the student becomes an official member of the class. The student can then access the class schedule, attendance information, learning results, tuition information and related notifications. Otherwise, the system updates the enrollment status to REJECTED and sends a notification containing the reason for rejection so that the student may

consider another suitable class.

b, Business process of Student Registration and Class Enrollment

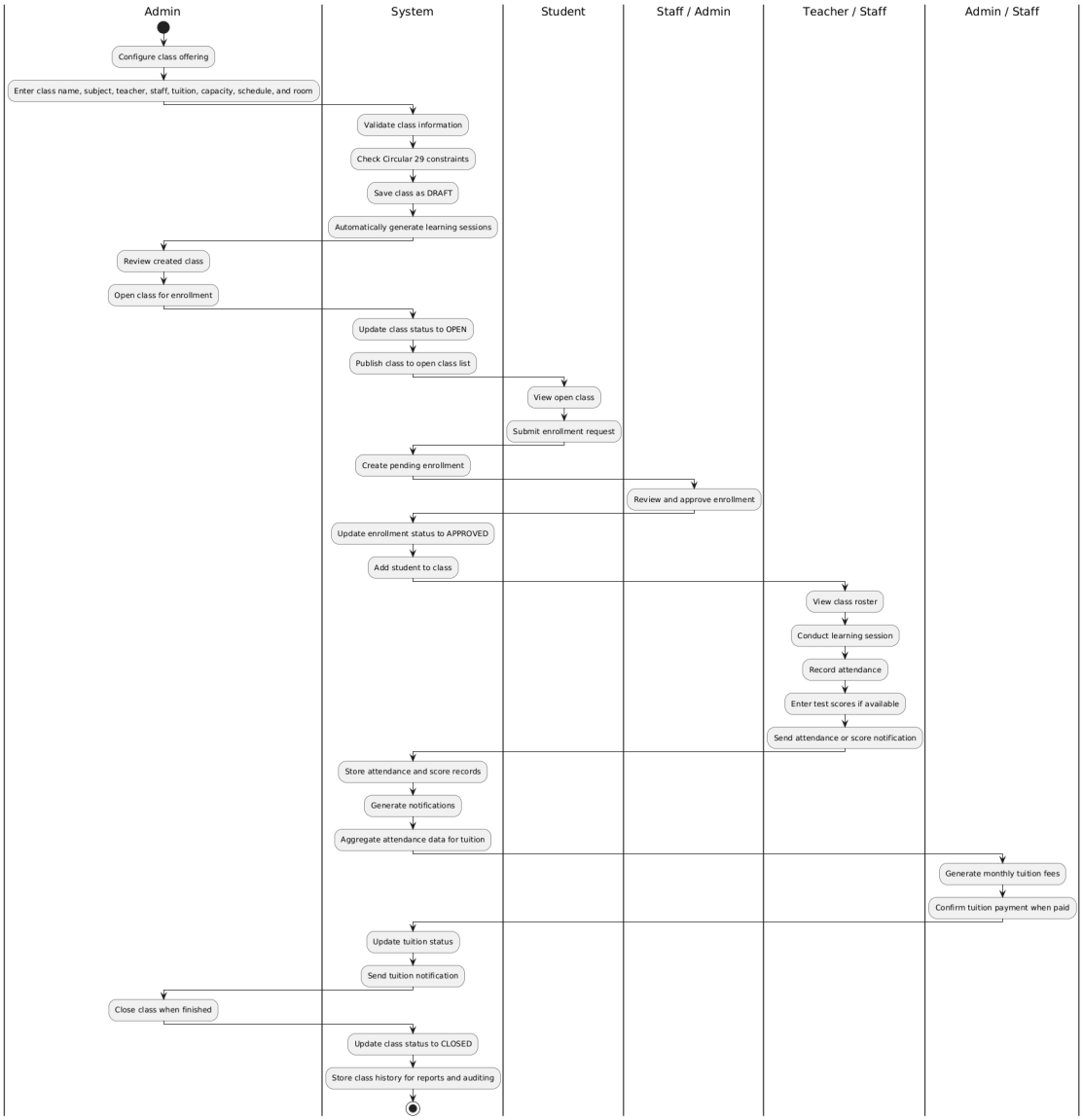


Figure 2.6: Business process of Student Registration and Class Enrollment

Figure 2.6 describes the business process of class organization and learning management at the tutoring center. The process begins when the administrator configures a new class offering by specifying information such as the class name, subject, teacher in charge, supporting staff, class schedule, tuition fees and the maximum number of students. After the class information is saved successfully, the system validates the configured information and automatically generates teaching sessions based on the defined schedule. The administrator can then change the class status to OPEN so that students are allowed to submit enrollment requests. During class operation, teachers and staff members use the system to view the class roster, conduct teaching sessions, record attendance, enter learning results and publish class-related notifications when necessary. The system stores attendance and

learning records, generates notifications and aggregates attendance data to support monthly tuition processing. When tuition records are generated, staff members or administrators can review and confirm payments when necessary. The system updates the tuition status and sends corresponding notifications to students. After the course has been completed, the administrator closes the class offering and all information related to the class, teaching sessions, attendance records, learning results and tuition records is retained for future lookup, reporting and auditing purposes.

2.3 Functional description

2.3.1 Access and Maintain Account Use Case Specification

Table 2.1 presents the detailed specification of the Access and Maintain Account use case. This is a foundational use case of the system that allows users to access the system through personal accounts, authenticate login credentials and manage their personal profiles. For students, the system supports new account registration so that they can participate in learning activities at the tutoring center. For teachers, staff members and administrators, accounts are created in advance by the administrator and users can log in, update personal information and change their passwords when necessary.

Table 2.1: Access and Maintain Account Use Case Specification

Section	**Content**
Use case ID	UC01
Use case name	Access and Maintain Account
Actors	Student, Teacher, Staff, Admin
Description	This use case allows users to register an account (for Students), log into the system, access functions according to their roles, update personal profile information and change passwords when necessary.
Preconditions	The user has an Internet-connected device and can access the system. For login, the user has a valid and active account. For profile updates or password changes, the user has successfully logged into the system.

Section	**Content**
Main flow of events	1. The user accesses the system. 2. If the user is a student without an account, the user selects the registration function and enters the required information. 3. The system validates the input data and creates a new student account. 4. The user selects the login function. 5. The system displays the login form. 6. The user enters a username and password. 7. The system authenticates the login credentials and determines the user role. 8. The system creates a login session and redirects the user to the interface corresponding to the assigned role. 9. The user accesses the profile page. 10. The system displays the account and profile information. 11. The user updates personal information or changes the password. 12. The system validates the input data and saves the changes. 13. The system displays a successful update notification.
Alternative flows of events	1. If the registration data is invalid or the username already exists, the system displays an error message and requests the user to re-enter the information. 2. If the username or password is missing, the system requests the user to provide complete information. 3. If the username or password is incorrect, the system denies login and displays an error message. 4. If the account is inactive, the system does not allow the user to log in. 5. If the current password is incorrect during a password change, the system rejects the update. 6. If profile information is invalid, such as an improperly formatted email address or phone number, the system displays an error message and requests correction.

Section	Content
Postconditions	The user successfully logs in and receives access rights corresponding to their role. If profile information or passwords are updated successfully, the new information is stored in the database. If the operation fails, the existing data remains unchanged.

Table 2.2 presents the primary input data for the Access and Maintain Account use case. This information is utilized during account registration, login, personal profile updates and password changes.

Table 2.2: Input Data for Account Access and Management

No.	Input Data	Notes	Required	Example
1	Username	Used to register or log into the system	Yes	nguyenvana
2	Password	Used to authenticate the user account	Yes	Abc@123456
3	Email	Stored in account and user profile information	No	nguyenvana@gmail.com
4	Full name	Identification information in the personal profile	Yes	Nguyễn Văn A
5	Phone number	User contact information	No	0912345678
6	Date of birth	Personal profile information	No	2007-03-15
7	Gender	Personal profile information	No	Male
8	Address	Only applicable to students and staff	No	Hà Nội
9	Current school	Only applicable to students	Yes for Student	THPT Đan Phượng

No.	Input Data	Notes	Required	Example
10	Grade level	Only applicable to students	Yes for Student	11
11	Class at school	Only applicable to students	No	11A1
12	Parent phone number	Only applicable to students	Yes for Student	0987654321
13	Current workplace	Only applicable to teachers	Yes for Teacher	THPT Đan Phượng
14	Professional qualification	Only applicable to teachers	No	Master's degree
15	Teaching specialization	Only applicable to teachers	No	Mathematics
16	List of current teaching classes at school	Used to support compliance verification with Circular No. 29/2024/TT-BGDĐT	No	11A1, 11A2
17	Current password	Used when the user changes the password	Yes when changing password	Abc@123456
18	New password	Used when the user changes the password	Yes when changing password	New@123456
19	Confirm new password	Used to re-verify the new password	Yes when changing password	New@123456

2.3.2 Configure Class Offerings Use Case Specification

Table 2.3 presents the detailed specification for the Configure Class Offerings use case. This is one of the most critical use cases of the system, allowing administrators to create and configure new classes at the center. During the configuration process, the administrator can set class information such as the subject, assigned teacher, supporting staff, class schedule, tuition fee, maximum number of students

and other relevant information. Once the class is successfully created, the system automatically generates a list of class sessions based on the established schedule.

Table 2.3: Configure Class Offerings Use Case Specification

Section	**Content**
Use case ID	UC02
Use case name	Configure Class Offerings
Actors	Admin
Description	This use case allows administrators to create new classes, update and manage class information at the center. The system supports configuring the subject, assigned teacher, class schedule, tuition fee, maximum number of students and automatically generates a list of class sessions for the class.
Preconditions	The administrator has successfully logged into the system. The system already contains data regarding subjects, teachers and assigned staff members.

Section	**Content**
Main flow of events	1. The administrator selects the “Configure Class Offerings” function. 2. The system displays a list of existing classes and the function to create a new class. 3. The administrator chooses to create a new class or edit an existing one. 4. The system displays the class information input form. 5. The administrator enters the class details, including the class name, subject, assigned teacher, supporting staff, class schedule, tuition fee, maximum number of students and other relevant information. 6. The system validates the input data and verifies business constraints. 7. The system saves the class information in an initialized state. 8. The system automatically generates a list of class sessions based on the configured schedule. 9. The system displays a successful class creation or update notification. 10. The administrator can open the class to allow student registration.
Alternative flows of events	1. At step 6, if required information is missing, the system displays an error message and prompts the user to re-enter. 2. At step 6, if the assigned teacher has a schedule conflict with another class, the system rejects the save operation and displays a schedule conflict notification. 3. At step 6, if the class schedule or subject violates the regulations of Circular No. 29/2024/TT-BGDĐT, the system refuses to create the class and displays the corresponding error message. 4. At step 8, if the system cannot automatically generate the class sessions, it displays an error message and does not complete the class creation process.

Section	Content
Postconditions	The class information is saved to the database. The list of class sessions is automatically created and linked to the corresponding class. The class is saved in an initialized state and can be opened for registration later. If the class creation process fails, the system does not record any changes.

Table 2.4 presents the primary input data when the administrator performs new creation or updates class information.

Table 2.4: Input Data for Class Configuration

No.	Input Data	Notes	Required	Example
1	Class name	The identifying name of the class	Yes	Toán 11 Nâng cao 01
2	Subject	Selected from the list of available subjects	Yes	Toán học
3	Grade level	The target grade level of the class	Yes	11
4	Assigned teacher	Selected from the list of teachers	Yes	Nguyễn Văn B
5	Assigned staff member	Selected from the list of staff members	No	Trần Thị C
6	Tuition fee per session	The tuition rate applicable to each class session	Yes	120000
7	Maximum number of students	The limit on the number of participating students	Yes	30
8	Days of the week	Multiple study days can be selected	Yes	Thứ Hai, Thứ Năm
9	Start time	The start time of the class session	Yes	19:00

No.	Input Data	Notes	Required	Example
10	End time	The end time of the class session	Yes	21:00
11	Opening date	The start date of the class organizing process	Yes	2026-08-10
12	Closing date	The end date of the class	No	2026-12-20
13	Classroom	The location where the class is organized	No	Phòng A203
14	Class description	Additional information about the class	No	Lớp luyện thi học sinh giỏi

2.3.3 Register for Classes Use Case Specification

Table 2.5 presents the detailed specification for the Register for Classes use case. This use case allows students to search for open classes, view the detailed information of each class and submit a registration request to participate. During the registration process, the system verifies business conditions, such as the class status, the current number of students and the regulations of Circular No. 29/2024/TT-BGDĐT, before recording the registration request.

Table 2.5: Register for Classes Use Case Specification

Section	Content
Use case ID	UC03
Use case name	Register for Classes
Actors	Student
Description	This use case allows students to view the list of currently open classes, look up detailed information and submit a registration request to join a class that matches their learning needs.
Preconditions	The student has successfully logged into the system. The system has at least one class currently in an open registration state.

Section	**Content**
Main flow of events	1. The student selects the "Register for Classes" function. 2. The system displays a list of classes currently open for registration. 3. The student searches for or selects a class of interest. 4. The system displays the detailed information of the class, including the subject, assigned teacher, class schedule, tuition fee and current number of students. 5. The student selects the function to register for the class. 6. The system verifies the registration conditions and business constraints. 7. If the request is valid, the system records the registration request in a pending approval state. 8. The system sends a confirmation notification that the registration request has been received. 9. The student can track the status of the registration request within the system.
Alternative flows of events	1. At step 2, if no classes are currently open for registration, the system displays a corresponding notification. 2. At step 6, if the class has reached its maximum student capacity, the system rejects the registration request and displays an error message. 3. At step 6, if the student has already submitted a registration request or is already participating in the class, the system does not allow duplicate registration. 4. At step 6, if the registration request violates the regulations of Circular No. 29/2024/TT-BGDĐT, the system rejects the registration request and displays a message explaining the reason. 5. At step 7, if an error occurs during the data saving process, the system displays an unsuccessful registration notification.

Section	Content
Postconditions	If the registration request is valid, the system creates a registration record in a pending approval state and sends a confirmation notification to the student. If the registration process is unsuccessful, the system does not record any changes to the registration data of the student.

Table 2.6 presents the primary input data when students perform class registration.

Table 2.6: Input Data for Class Registration

No.	Input Data	Notes	Required	Example
1	Search keyword	Search by class name or subject	No	Toán lớp 11
2	Selected class	The class that the student wishes to register for	Yes	Toán 11 Nâng cao 01
3	Subject	Displays subject information of the class	Yes	Toán học
4	Assigned teacher	Displays teacher information of the class	Yes	Nguyễn Văn B
5	Class schedule	Displays the days and hours of the class sessions	Yes	Thứ Hai, Thứ Năm, 19:00–21:00
6	Tuition fee per session	Displays the tuition rate of the class	Yes	120000 VNĐ
7	Current number of students	Displays the class capacity status	Yes	24/30
8	Registration confirmation	The student confirms submitting the registration request	Yes	Agree

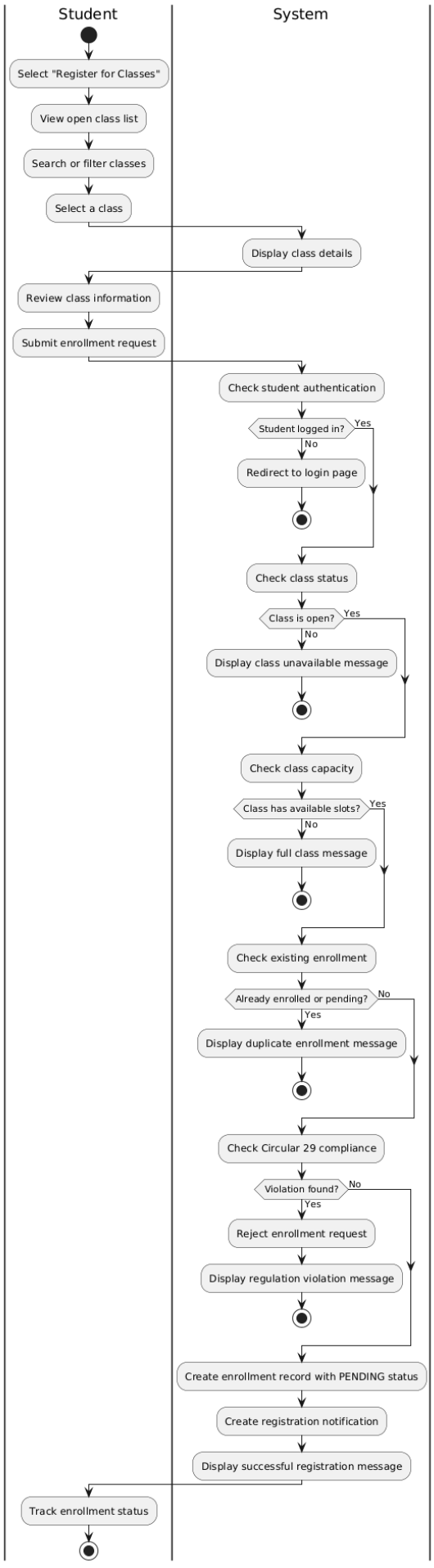


Figure 2.7: Activity diagram for the Register for Classes use case

Figure 2.7 illustrates the activity diagram for the Register for Classes use case. The process initiates when the student accesses the list of open classes, searches for a suitable class and views the detailed information before submitting a registration request. After the student confirms the registration, the system sequentially verifies the class status, the number of remaining vacancies, the previous registration status of the student and constraints related to Circular No. 29/2024/TT-BGDĐT. If the registration request is valid, the system creates a registration record with a pending approval status and sends a confirmation notification to the student. If one of the conditions is not satisfied, the system rejects the request and displays a corresponding error message.

2.3.4 Process Enrollment Requests Use Case Specification

Table 2.7 presents the detailed specification for the Process Enrollment Requests use case. This use case allows center staff members or administrators to review class registration requests submitted by students, subsequently approving or rejecting them based on student information, class details, remaining vacancies and related business constraints. This is a critical intermediary step that helps the center control the official student roster of each class.

Table 2.7: Process Enrollment Requests Use Case Specification

Section	**Content**
Use case ID	UC04
Use case name	Process Enrollment Requests
Actors	Staff, Admin
Description	This use case allows center staff members or administrators to process class registration requests currently in a pending approval state. The processor can verify registration details, approve valid requests, or reject requests with specific reasons provided.
Preconditions	The Staff or Admin has successfully logged into the system. The system contains at least one class registration request in a pending approval state. For Staff, the registration request must belong to a class that the staff member is assigned to manage.

Section	**Content**
Main flow of events	1. The Staff or Admin selects the “Process Enrollment Requests” function. 2. The system displays a list of registration requests waiting for approval. 3. The processor selects a registration request to view its details. 4. The system displays student information, class details, the assigned teacher, the class schedule, and the current status of the request. 5. The processor reviews the registration details and selects to approve the request. 6. The system re-verifies the request status, class capacity, and processing permissions of the user. 7. If the request is valid, the system updates the registration status to approved. 8. The system updates the current number of students in the class. 9. The system sends an approval notification to the student. 10. The system displays a successful processing notification.

Section	**Content**
Alternative flows of events	1. At step 2, if there are no pending requests, the system displays an empty list notification. 2. At step 5, if the processor chooses to reject the request, the system requires inputting a rejection reason, updates the registration status to rejected, and sends a notification to the student. 3. At step 6, if the registration request is no longer in a pending approval state, the system prohibits reprocessing and displays an error message. 4. At step 6, if the class has reached its maximum capacity, the system does not allow request approval and displays a class full notification. 5. At step 6, if the Staff is not assigned to manage the corresponding class, the system rejects the operation due to insufficient access permissions. 6. If an error occurs during the data update process, the system displays a processing failure message and retains the current registration status.
Postconditions	If the request is approved, the registration record is updated to the APPROVED status, the current number of students in the class increases and the student receives a corresponding notification. If the request is rejected, the registration record is updated to the REJECTED status along with the rejection reason. If the operation fails, the registration data and class information remain unchanged.

Table 2.8 presents the primary input data when a Staff or Admin processes student class registration requests.

Table 2.8: Input Data for Processing Registration Requests

No.	Input Data	Notes	Required	Example
1	Registration request	The pending registration record selected for processing	Yes	ENR00025
2	Student information	Full name, school, grade level, class at school and parent phone number	Yes	Nguyễn Văn A, lớp 11A1
3	Class details	Class name, subject, teacher, schedule and current number of students	Yes	Toán 11 Nâng cao 01
4	Processing decision	Approve or reject the registration request	Yes	Approve
5	Rejection reason	Mandatory when the processor selects to reject the request	Yes when rejecting	The class schedule no longer aligns with the student's personal schedule
6	Processor	The Staff or Admin performing the request processing operation	Yes	Trần Thị C

2.3.5 Monitor Teaching Activities Use Case Specification

Table 2.9 presents the detailed specification for the Monitor Teaching Activities use case. This use case allows teachers and center staff members to monitor the class organization process after students have been approved to participate. The primary activities include viewing the class roster, viewing the list of class sessions, tracking students within the class, taking attendance and updating learning results if applicable.

Table 2.9: Monitor Teaching Activities Use Case Specification

Section	**Content**
Use case ID	UC05
Use case name	Monitor Teaching Activities
Actors	Teacher, Staff
Description	This use case allows teachers and center staff members to monitor class teaching activities, including student rosters, class schedules, class sessions, attendance status and student academic results. Teachers can perform attendance tracking and update test scores, while supporting staff members monitor the operational progress of the class.
Preconditions	The Teacher or Staff has successfully logged into the system. The class has been created and contains a list of class sessions. For a Teacher, the class must belong to the currently logged-in teacher. For a Staff member, the class must fall within the management scope of that specific staff member.

Section	**Content**
Main flow of events	1. The Teacher or Staff selects the “Monitor Teaching Activities” function. 2. The system displays a list of classes within the access scope of the user. 3. The user selects a specific class to view details. 4. The system displays class information, the roster of approved students, and the list of class sessions. 5. The Teacher selects a class session that requires monitoring or attendance tracking. 6. The system displays the student roster of the class for the corresponding class session. 7. The Teacher updates the attendance status of each student, such as present, absent, tardy, or excused absence. 8. The Teacher enters notes or test scores if applicable. 9. The system validates the input data and stores the attendance information and scores into the database. 10. The system updates the class session status and sends notifications related to attendance or academic results to the students if necessary. 11. The Teacher or Staff reviews the updated data to track the academic progress of the class.

Section	Content
Alternative flows of events	1. At step 2, if the user has no classes within their access scope, the system displays an empty list notification. 2. At step 3, if the user selects a class outside their authorization scope, the system denies access. 3. At step 6, if the class does not have any approved students yet, the system displays a notification stating that there are no students in the class. 4. At step 9, if the attendance data or scores are invalid, the system displays an error message and prompts for corrections. 5. At step 9, if the attendance record of the student for that class session already exists, the system updates the existing record instead of creating a duplicate entry. 6. If an error occurs during the data saving process, the system displays an update failure notification and retains the previous data.
Postconditions	Attendance information, session notes and student test scores are saved to the system. This data can be utilized by students to track their learning progress, by staff and administrators to monitor class operations and serves as a supportive basis for tuition calculations and subsequent reporting.

Table 2.10 presents the primary input data when a Teacher or Staff monitors and updates the teaching activities of a class.

Table 2.10: Input Data for Monitoring Teaching Activities

No.	Input Data	Notes	Required	Example
1	Class	The class selected for monitoring	Yes	Toán 11 Nâng cao 01
2	Class session	The specific class session to view or track attendance	Yes	Class session on 2026-08-12

No.	Input Data	Notes	Required	Example
3	Student roster	The roster of students approved to participate in the class	Yes	Nguyễn Văn A, Trần Thị B
4	Attendance status	The participation status of each student in the session	Yes when taking attendance	Present
5	Attendance notes	Additional notes for the attendance status	No	Excused absence due to illness
6	Test score	The academic score of the student in the session if a test is administered	No	8.5
7	Academic performance remark	Additional evaluation comments from the teacher	No	Performed well on the test
8	Related notifications	Notification content sent to students when necessary	No	Test results have been updated

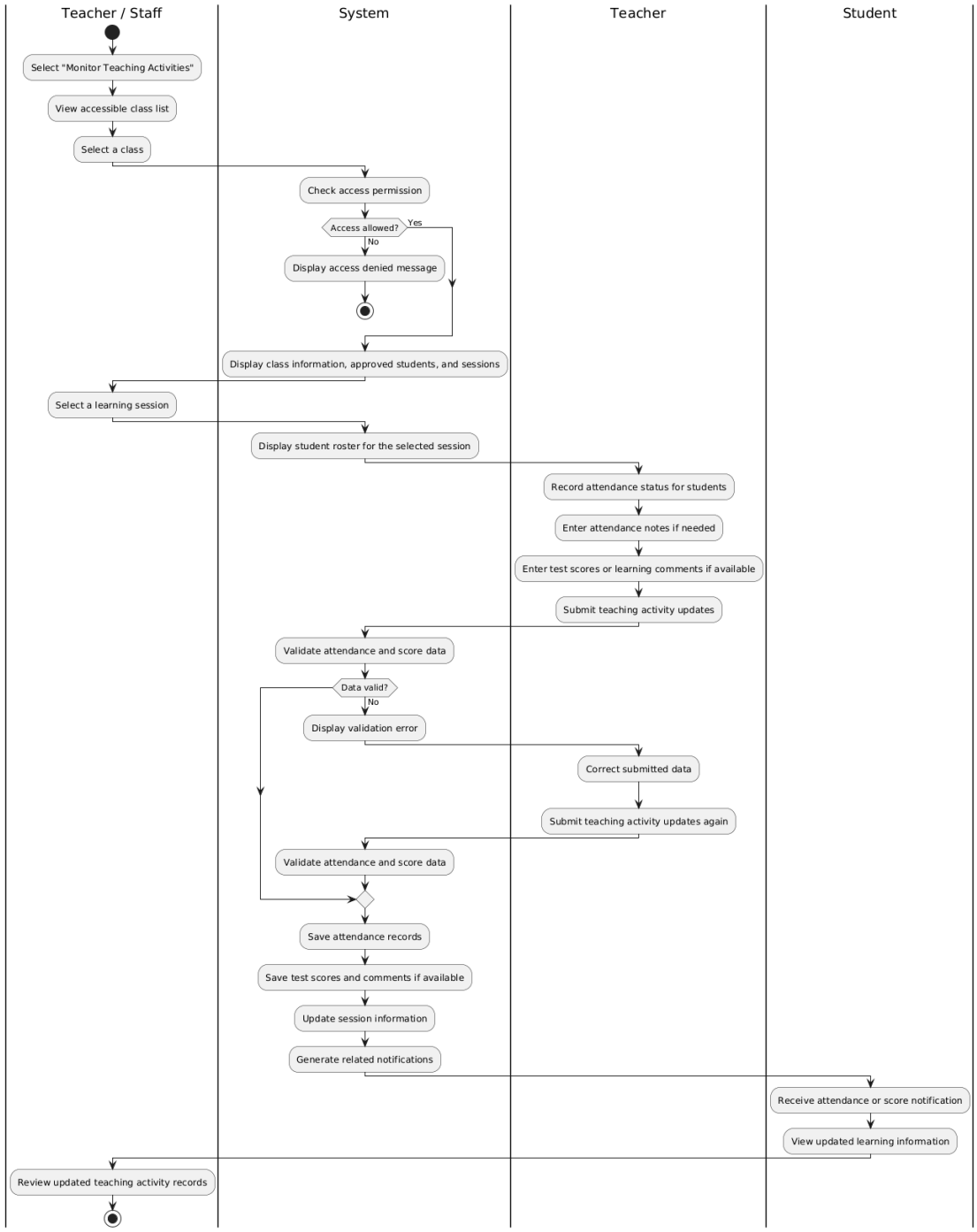


Figure 2.8: Activity diagram for the Monitor Teaching Activities use case

Figure 2.8 illustrates the activity diagram for the Monitor Teaching Activities use case. The process initiates when a teacher or a center staff member accesses the list of classes within their scope of responsibility and selects a specific class to monitor. After the system verifies access permissions, the user can view class information, the roster of approved students and the corresponding class sessions. The teacher subsequently selects a class session to update the attendance status, notes and test scores if applicable. The system validates the data accuracy, stores the related records and generates notifications so that students can track their learning

progress on the system.

2.3.6 Process Tuition Records Use Case Specification

Table 2.11 presents the detailed specification for the Process Tuition Records use case. This use case allows Staff or Admin to monitor and process student tuition information. The system supports viewing the tuition list by month, class and payment status, while enabling Staff or Admin to confirm collection when a student or parent pays cash at the center or when the payment status has not been accurately updated by the system.

Table 2.11: Process Tuition Records Use Case Specification

Section	**Content**
Use case ID	UC06
Use case name	Process Tuition Records
Actors	Staff, Admin
Description	This use case allows Staff or Admin to view the student tuition roster, track payment statuses and confirm cash collection when necessary.
Preconditions	The Staff or Admin has successfully logged into the system. The system already contains data regarding classes, students and corresponding tuition. For Staff, the tuition data must belong to the classes that the staff member is assigned to manage.

Section	**Content**
Main flow of events	1. The Staff or Admin selects the "Process Tuition Records" function. 2. The system displays the tuition roster accompanied by filters for month, class, and payment status. 3. The Staff or Admin selects the required search criteria. 4. The system displays the tuition roster matching the search conditions along with the corresponding payment information. 5. If the student or parent has paid in cash, or if the payment status has not been accurately updated by the system, the Staff or Admin selects the collection function. 6. The Staff or Admin enters the payment information required for confirmation, including the amount paid and notes if applicable. 7. The system validates the data and updates the corresponding tuition information. 8. The system sends a notification of the tuition processing results to the student and displays a successful processing message to the Staff or Admin.
Alternative flows of events	1. At step 4, if no tuition data matches the search conditions, the system displays an empty list notification. 2. At step 5, if the tuition record has already been paid in full, the system prohibits the collection operation. 3. At step 6, if the payment information is invalid, the system displays an error message and prompts for a reentry. 4. At step 7, if the Staff lacks execution permissions for the corresponding class, the system denies the operation and displays an insufficient permissions notification. 5. At step 7, if an error occurs during the data update process, the system displays a processing failure message and retains the previous tuition information.

Section	Content
Postconditions	Tuition information is displayed or updated according to the actions of the Staff or Admin. If the collection is confirmed successfully, the tuition status and related payment details are updated in the system and the student receives a corresponding notification.

Table 2.12 presents the primary input data when a Staff or Admin monitors and processes student tuition.

Table 2.12: Input Data for Tuition Processing

No.	Input Data	Notes	Required	Example
1	Tuition month	Used to filter the tuition roster by month	No	2026-08
2	Class	Used to filter the tuition roster by class	No	Toán 11 Nâng cao 01
3	Payment status	Used to filter the tuition roster by payment state	No	Chưa nộp
4	Amount paid	Entered when the Staff or Admin confirms collection	Yes when collecting	840000
5	Payment notes	Additional notes when confirming collection	No	Phụ huynh nộp tiền mặt tại trung tâm
6	Confirming person	The Staff or Admin performing the collection operation	Automatically recorded by the system	Trần Thị C

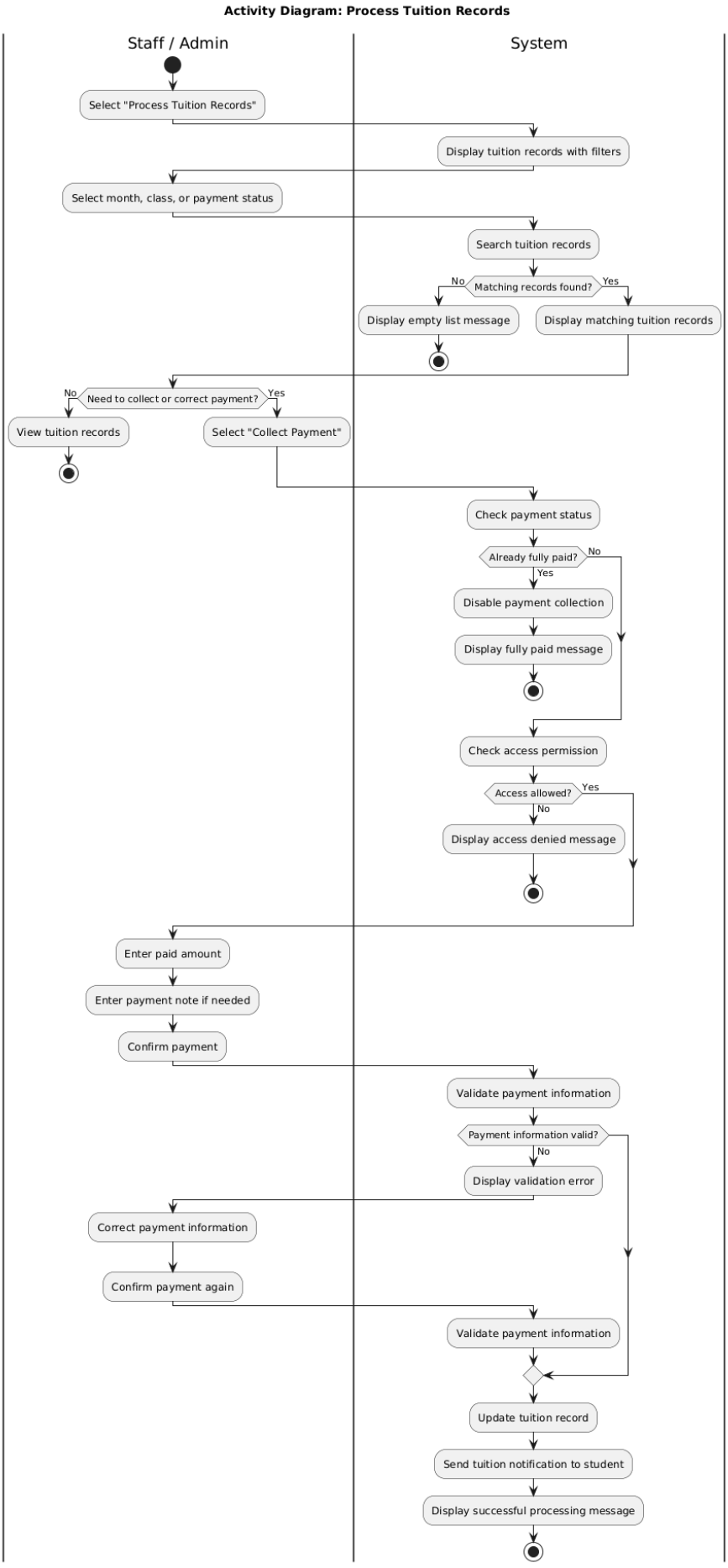


Figure 2.9: Activity diagram for the Process Tuition Records use case

Figure 2.9 illustrates the sequence diagram for the Process Tuition Records use case. The process begins when a staff member or administrator selects a specific month for tuition processing. The system then retrieves the completed class sessions, approved enrollments and attendance records to determine the number of billable sessions for each student. Based on the calculated results, the system creates or updates monthly tuition records and returns the generated tuition information to the user interface. Finally, the tuition list, including payment status and outstanding amounts, is displayed to support subsequent payment processing and tracking.

2.4 Non-functional requirements

2.4.1 Usability

The system is designed for different user groups, including students, teachers, staff members and administrators. Parents are treated as related stakeholders rather than direct system users because they do not have separate accounts in the current version. Parent contact information is stored through the guardian phone number in the student profile. Notifications and tuition-related reminders that may concern parents are simulated as in-app notifications and are displayed through the student account.

2.4.2 Reliability and Security

The system stores critical information such as personal data of students and teachers, class information, tuition fees and academic results. The platform must guarantee accurate, consistent data management and restrict data access solely to authorized users. Critical business operations - including class enrollment, attendance tracking and tuition management - must strictly adhere to defined business rules. Data undergoes comprehensive validity validation prior to storage to mitigate discrepancies or inconsistencies during system operation.

2.4.3 Maintainability and Scalability

The operations of tutoring centers fluctuate based on growth scale, management requirements and educational regulatory standards. The system design must prioritize ease of maintenance and long-term scalability. Adding new functions, adjusting business workflows, or updating management policies must execute without major disruptions to active system operations. This architectural approach ensures the software consistently satisfies the long-term operational demands of the center.

This chapter has presented the requirement survey and analysis for the EduCenter system. From the limitations of manual management methods and the practical operational needs of tutoring centers, the chapter identified the main functional

groups of the system, the participating actors, several important business processes and six representative use cases for detailed specification. The non-functional requirements were also defined as a basis for architecture design, database design and technology selection in the following chapters.

CHAPTER 3. THEORETICAL BACKGROUND AND TECHNOLOGIES

This chapter presents the theoretical background and technologies adopted in the development of the EduCenter system. It first discusses the frontend technologies used to build the user interface, including TypeScript, React, Next.js, Tailwind CSS and Zustand state management. The chapter then introduces the backend technologies and theoretical foundations that support system implementation, including Node.js, Express.js, RESTful API architecture, JSON Web Token authentication, role-based access control and the MySQL relational database management system. For each technology, the chapter explains its fundamental concepts, compares it with possible alternatives and justifies its selection in the context of the EduCenter project.

3.1 Frontend Technologies

3.1.1 TypeScript

TypeScript [[5]] is a programming language built on JavaScript that adds a static type system to support the development of large-scale applications. This language allows developers to define data types for variables, functions and complex data structures. It also supports automatic type inference and compile-time error checking. After compilation, TypeScript source code is converted into JavaScript and can execute on any modern browser as well as the Node.js environment.

In web interface development, developers can choose between vanilla JavaScript and TypeScript. JavaScript offers advantages in flexibility and accessibility. However, the lack of a static type-checking mechanism can increase risks as the project scale and the number of interacting components grow. Conversely, TypeScript enhances source code consistency, supports early error detection and improves application maintainability.

For the EduCenter system, the user interface serves multiple user groups, including administrators, staff, teachers and students. The system is organized into several business modules such as class management, enrollment, attendance, tuition and notifications, while frequently exchanging data with the backend via RESTful APIs. These modules utilize multiple closely related data models and must maintain consistency during data transmission and reception. TypeScript was selected to increase source code safety, minimize errors caused by incorrect data types and support the long-term development, expansion and maintenance of the system.

3.1.2 React and Next.js

React [[6]] is an open-source JavaScript library developed by Meta to build user interfaces based on a component-based architecture. Each interface component is constructed as an independent unit capable of managing its own state and being reused. This approach makes developing applications with a large number of screens and functions more convenient, while enhancing the maintainability and scalability of the source code.

Next.js [[7]] is a framework developed on top of React that provides multiple mechanisms to support building modern web applications, such as file-system-based routing, modular source code organization, compilation and deployment optimization and support for combining server-side and client-side components. Consequently, Next.js reduces the initial configuration workload and creates a clear project structure suitable for medium and large-scale applications.

In current web interface development, popular frameworks and libraries that can be utilized include Angular, Vue.js, or React combined with Next.js. Among the available frontend technologies, Angular provides a comprehensive ecosystem but requires more conventions and configuration. Vue.js offers simplicity and accessibility, yet its ecosystem is less dominant in large-scale applications. In contrast, React and Next.js provide a balanced combination of flexibility, scalability and development efficiency, making them suitable for the requirements of the EduCenter system.

For the EduCenter system, the user interface includes various business screens such as user management, class configuration, enrollment, attendance, tuition, notifications and statistics. The system also serves multiple target user groups with different permissions and operational workflows. Constructing the interface according to the React component model enhances the reusability of shared components such as forms, data tables, dialog boxes and navigation elements. Simultaneously, Next.js provides a clear routing organization mechanism and supports the segregation of functional areas within the system, such as authentication zones and logged-in user zones. The combination of React and Next.js helps the system maintain a consistent source code structure, eases maintenance and facilitates future functional expansions.

3.1.3 Tailwind CSS and Component-Based User Interface

Tailwind CSS [[8]] is a utility-first CSS framework that provides a collection of predefined classes to construct user interfaces directly within the source code of the component. Instead of building separate CSS files with a large number of custom

classes, developers can combine utility classes to describe layouts, spacing, colors, dimensions and display states of individual interface components. This approach improves development consistency and allows interface styles to be composed directly within reusable components while maintaining a uniform appearance across the application.

During web interface development, alternative styling solutions that can be utilized include vanilla CSS, CSS Modules, or styled-components. Vanilla CSS is simple and accessible but can lead to dispersed code and difficulties in reusability as the project scale grows. CSS Modules and styled-components support scoping the application of CSS, yet they require multiple configuration files and more complex code organization. Conversely, Tailwind CSS offers a lightweight, flexible approach that is particularly suited for applications built on a component-based architecture.

The EduCenter system comprises various business screens with highly repetitive interface elements such as navigation bars, sidebars, input forms, data tables, confirmation dialogs and information cards. The system must also serve multiple user groups on devices with different screen sizes. The user interface is therefore constructed following a component-based model where each element is packaged into an independent unit capable of being reused across multiple screens and functions. Combining React components with Tailwind CSS accelerates interface construction, maintains display consistency and facilitates the long-term maintenance and expansion of the system.

3.1.4 Zustand State Management

State management is a critical issue in modern web application development, particularly for systems with multiple screens and multiple user groups. Centralized state management enables interface components to share and synchronize data while limiting data propagation through multiple component levels, which increases source code complexity.

Currently, multiple state management solutions exist for React applications, such as React Context, Redux Toolkit, Recoil, Jotai and Zustand. Redux Toolkit provides a strict state management architecture and is suitable for applications with a large volume of global states. However, it requires a significant amount of boilerplate code and relatively complex configuration. React Context offers a simpler approach but can increase the number of interface re-renders when the state changes. Conversely, Zustand [[9]] is a lightweight state management library with a simple API, minimal configuration code and easy integration with React applications.

For the EduCenter system, the majority of business data is retrieved directly from the backend via RESTful APIs and only a small amount of information needs to be shared across screens. This shared data primarily relates to user authentication status, including account information, user profiles and access tokens. Using a highly complex state management solution is therefore unnecessary. Within the scope of this thesis, Zustand was selected to manage the authentication state on the frontend, allowing user login information to be stored and maintained throughout the session while simplifying source code development and maintenance. This choice aligns with the current scale of the system and facilitates future expansion as the number of functions and users continues to grow.

3.2 Backend Technologies and Theoretical Foundations

3.2.1 Node.js and Express.js

Node.js [[10]] is a server-side JavaScript runtime environment that enables the use of the JavaScript language to construct backend applications and network services. Due to its asynchronous processing mechanism and event-driven model, Node.js efficiently handles input/output (I/O) operations while supporting the development of web applications with a large number of concurrent access requests.

Express.js [[11]] is a web framework built on top of Node.js that provides a collection of components to support server-side application development, such as request routing, data processing middleware, HTTP response management and business module-based API organization. Compared to building applications directly with native Node.js HTTP libraries, Express.js significantly reduces the volume of source code required for implementation and establishes a clear, maintainable development structure.

During backend development for web applications, alternative technologies that can be utilized include Spring Boot, ASP.NET Core, or Django. These frameworks offer multiple integrated features and are suitable for systems with specific, distinct requirements. However, they typically require developers to work with more complex conventions and configuration mechanisms. Conversely, Node.js combined with Express.js offers a relatively simple, flexible approach that allows for the rapid development of REST services.

For the EduCenter system, the backend must process various business operations, including user account management, class management, enrollment, attendance, tuition, notifications and statistics. Most system functions involve data management operations and business workflows such as account management, class configuration, enrollment processing, attendance tracking, tuition management and

notifications, all of which frequently exchange data with the frontend through APIs. Concurrently, the system is developed across both the frontend and backend using the JavaScript/TypeScript language, which reduces language discrepancies and enhances consistency during the development process. Node.js and Express.js were therefore selected to construct a backend with a simple structure that is easy to develop, maintain and satisfy the operational requirements of the EduCenter tutoring center management system.

3.2.2 RESTful API Architecture

A RESTful API (Representational State Transfer Application Programming Interface) [[12]] is an architectural style widely utilized in constructing modern web applications. Under this architecture, the system is organized into resources and provides programming interfaces via the HTTP protocol to execute operations such as querying, creating, updating, or deleting data. Data exchanged between system components is typically represented in JSON format, which reduces transmission overhead and facilitates integration across disparate platforms.

In current web application development, alternative approaches alongside RESTful APIs include SOAP or GraphQL. SOAP provides a strict data exchange mechanism and supports multiple extension standards, yet its implementation and message processing are relatively complex. GraphQL allows the client to actively select the specific data to be retrieved but requires a more intricate schema definition and query handling mechanism. Conversely, RESTful APIs offer a simple structure, ease of implementation, straightforward testing and suitability for most information management applications based on CRUD operations.

For the EduCenter system, the user interface and business logic processing are developed separately and must frequently exchange data over the network. Most system functions, such as login, user management, class configuration, enrollment, attendance, tuition management and notifications, are implemented as business resources and accessed through distinct APIs. Utilizing RESTful APIs enables the frontend and backend to be developed, tested and deployed relatively independently, while establishing a unified communication mechanism for the entire system. Furthermore, the REST architecture facilitates future system expansion, such as developing mobile applications or integrating with other educational management systems without substantially modifying the existing business logic.

3.2.3 JSON Web Token Authentication and Role-Based Access Control

Authentication and authorization are two critical requirements for management systems with multiple user groups. Authentication is the process of verifying the

identity of a user before granting access to the system, whereas authorization determines the scope of resources and functions that the user is permitted to utilize after successful login.

JSON Web Token (JWT) [[13]] is an open standard widely used in modern web applications to implement stateless authentication. Upon successful login, the system generates a token containing the necessary identification information and transmits it back to the client. In subsequent requests, the client attaches this token to each request so the server can verify validity and determine the identity of the user. This approach reduces reliance on server-side session storage mechanisms and facilitates the development of systems where the frontend and backend are deployed independently.

Alongside JWT, alternative authentication mechanisms commonly utilized include session-based authentication or the OAuth2 and OpenID Connect protocols. Session-based authentication offers a relatively simple implementation but requires the server to maintain user session states. OAuth2 and OpenID Connect are suitable for systems requiring third-party login integration or federated identity implementation, yet their deployment and management are more intricate. Conversely, JWT features a lightweight structure, ease of implementation and particular compatibility with systems using a RESTful API architecture.

Regarding authorization mechanisms, prevalent models include Discretionary Access Control (DAC), Mandatory Access Control (MAC)and Role-Based Access Control (RBAC). Among these, RBAC [[14]] is an authorization model based on user roles, which groups access permissions according to specific roles rather than managing permissions for individual users. This approach simplifies access management while enhancing system consistency and scalability.

The EduCenter system serves multiple user groups, including administrators, staff members, teachers and students. Each user group possesses a distinct scope of functions and data access permissions. Parents are treated as related stakeholders rather than direct system users because they do not possess independent accounts in the current version. Parent contact information is maintained through the guardian phone number stored in the student profile and notifications or reminders that may concern parents are simulated as in-app notifications displayed through the student account. Students are only permitted to access information associated with their own learning activities, whereas staff members and administrators are authorized to perform management operations such as class configuration, enrollment processing and tuition management.

3.2.4 MySQL Relational Database Management System

A Relational Database Management System (RDBMS) is a foundational platform widely utilized in information management systems, wherein data is organized into tables interconnected through primary keys and foreign keys. This organizational approach ensures structured data storage, reduces redundancy and guarantees data consistency during retrieval and update operations.

MySQL [[15]] is an open-source relational database management system extensively employed in web application development. MySQL supports the SQL query language, transactional mechanisms, integrity constraints and efficient query processing for complex relational data. Consequently, MySQL is suitable for systems that require the storage of substantial volumes of structured data and frequently execute querying, updating, or statistical computation tasks.

Alongside MySQL, alternative data storage solutions such as PostgreSQL, SQLite, or MongoDB can be utilized. PostgreSQL provides advanced features and powerful scalability, yet its deployment and administration are typically more intricate. SQLite is suitable for small or local applications but faces significant limitations as the number of concurrent users increases. MongoDB is a non-relational database management system that offers flexible data storage in document format; however, managing complex data relationships and ensuring data integrity is less convenient than in relational database management systems.

For the EduCenter system, operational data comprises multiple tightly coupled entities, such as users, students, teachers, parents, classes, class sessions, enrollments, attendance, tuition and notifications. Many system functions require executing queries that combine data from multiple tables while ensuring consistency in critical business workflows, such as class enrollment, attendance tracking, or tuition management. MySQL was therefore selected as the database management system to leverage its relational data management capabilities, integrity constraints and transactional mechanisms. This choice ensures that the system data remains accurate, consistent and well-positioned for future expansion.

This chapter has presented the theoretical background and the main technologies employed in the development of the EduCenter system. The selected technologies provide a suitable combination of maintainability, development efficiency, scalability and support for the functional requirements of tutoring center management, including class organization, enrollment processing, teaching activities, tuition management and notifications. The concepts and technologies discussed in this chapter serve as the foundation for the architectural design and implementation

decisions presented in the following chapter.

CHAPTER 4. DESIGN, IMPLEMENTATION, AND EVALUATIONS

This chapter presents the design, implementation and evaluation of the EduCenter system. It begins with the selection of the software architecture and the overall package organization of both the frontend and backend components. The chapter then describes the detailed design of the user interface, core classes, communication flows and database structure. Following the design phase, the implementation results are presented through the technologies, tools and key functionalities developed during the project. Finally, the chapter evaluates the completed system through a set of functional test cases designed to verify the correctness of major business processes and regulatory constraints.

4.1 Architecture design

4.1.1 Software Architecture Selection

Based on the operational requirements of a tutoring center management system with diverse user groups, the EduCenter system is designed according to a Client-Server architecture model combined with a layered architecture, also referred to as a Layered Architecture or N-tier Architecture. The system utilizes a RESTful API as the primary communication mechanism between the interface and the server. This approach aligns with the specific characteristics of a web-based management application, wherein the user interface, business logic and data must be organized independently yet still coordinate closely throughout the entire processing lifecycle.

In the Client-Server model, the system is divided into two primary sides: the client and the server. The client is responsible for providing the interaction interface, receiving user actions and transmitting requests to the server via the HTTP protocol. The server receives these requests, executes business logic processing, interacts with the database and returns the results to the client in JSON data format. Separating these two sides increases flexibility in system development and maintenance because the frontend and backend can be modified, tested and deployed relatively independently.

Within the server side, the system is further organized into a layered architecture to clearly divide the responsibilities of each processing component group. Each layer undertakes a distinct role and only interacts directly with adjacent layers. This organizational structure reduces dependencies between components, limiting the mixing of interface handling, business logic and data management code in a single location. For a system with multiple operational workflows such as class

management, enrollment processing, attendance tracking, tuition and notifications, the layered architecture enhances source code readability and facilitates functional expansion.

In the EduCenter system, the presentation layer is constructed using Next.js and React, combined with Zustand for client-side state management. This layer encompasses user interfaces such as login pages, dashboard pages, class management pages, enrollment processing pages, tuition pages and notification pages. User actions on the interface are converted into RESTful API requests sent to the backend. The presentation layer does not directly handle complex business logic or access the database; instead, it focuses exclusively on displaying data, collecting input data and reflecting interaction states to the user.

The API layer is implemented using Express.js via routes and controllers. Routes define the API endpoints, HTTP methods and the chain of middleware to be applied to each function. Controllers receive requests, extract data from the body, parameters, or query strings and then call the corresponding service to process the business logic. At this layer, the system also applies an authentication mechanism using JSON Web Tokens and authorization based on user roles. Consequently, requests to the backend are verified before entering deeper into the business logic layer.

The business logic layer is implemented through services. This is the central layer of the system, housing core processing rules such as creating and updating classes, automatically generating class sessions from a fixed schedule, verifying class enrollment conditions, processing the approval or rejection of registrations, recording attendance, generating monthly tuition fees and creating internal notifications. A critical aspect of EduCenter is that rules related to Circular No. 29/2024/TT-BGDĐT are also positioned within the business logic layer. These include verifying whether a teacher is currently teaching a student at their main public school or checking conditions related to classes and schedules.

The data access layer is responsible for working with the MySQL relational database management system. In the current system, the backend primarily utilizes mysql2/promise and parameterized SQL queries to execute data operations. This layer handles reading, inserting, updating and deleting data within tables such as users, students, teachers, classes, enrollments, sessions, attendance, monthly_fees and notifications. Isolating database queries from the user interface and request dispatching components makes the system easier to control while mitigating the risk of duplicating data access logic in multiple places.

In addition to the layered architecture, the system incorporates security mechanisms using JWT and Role-Based Access Control. Upon successful user login, the backend generates a token containing identification data, roles and the corresponding user profile ID. This token is attached to subsequent requests by the frontend so that the backend can authenticate the user and verify access permissions. This approach is highly appropriate for EduCenter due to its multiple user roles, including students, teachers, staff and administrators, where each role is strictly permitted to execute a specific set of functions.

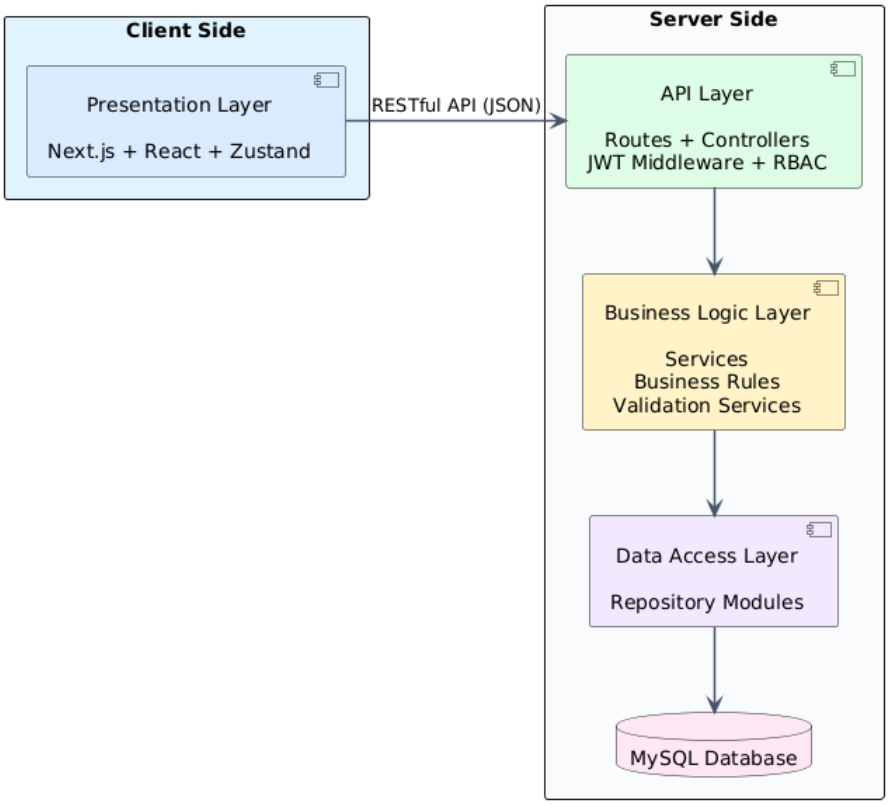


Figure 4.1: Layered Client-Server Architecture

Figure 4.1 describes how the layered architecture is applied to the EduCenter system. From top to bottom, user requests pass through the presentation layer, the API layer, the business logic layer, the data access layer and finally, the database layer. The response path flows in reverse from the database back to the user interface in the form of processing results or error notifications. This diagram demonstrates that the system is not organized according to a pure MVC model but fits a layered Client-Server architecture utilizing RESTful APIs.

Selecting a layered Client-Server architecture enables the EduCenter system to satisfy requirements regarding source code organization, maintainability and scalability. When a new function needs to be added, the development team can introduce the interface on the frontend, define the API within routes and controllers, append

the processing logic in services and then update the necessary data queries without altering the entire system. This architecture is also suitable for the scope of the thesis as it is sufficiently clear to manage complex business logic without incurring the heavy deployment and operational overhead associated with a microservices architecture.

4.1.2 Overall Design

Following the selection of the Client-Server architecture combined with the layered architecture, the EduCenter system is organized into two primary divisions: the frontend and the backend. Each division is partitioned into packages that possess distinct responsibilities and solely depend on lower-level or shared packages. This organizational structure minimizes dependencies between components, enhances source code reusability and facilitates system expansion and maintenance during the development process.

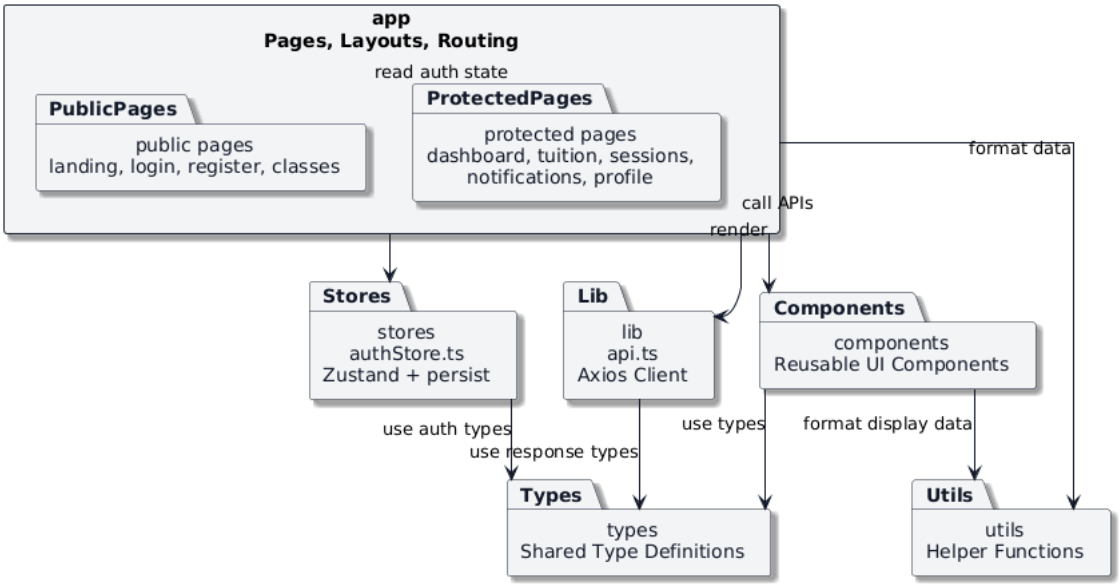


Figure 4.2: Frontend Package Diagram

Figure 4.2 describes the package diagram of the frontend. The frontend is constructed based on the Next.js App Router and is structured around the central app package, which is responsible for managing the pages, layouts and routing mechanisms of the application. This package is segregated into two primary page groups: PublicPages and ProtectedPages. Public pages encompass introductory, login, registration and class listing pages. Protected pages include the dashboard, tuition management, class session management, notifications and user profiles. This segregation controls access privileges based on the authentication status of the user while establishing a clearer navigation structure for the system.

The components package contains reusable user interface elements such as data

tables, input forms, information cards, dialog boxes and other display components. These elements are utilized across multiple pages within the system, thereby reducing source code duplication and ensuring user interface consistency.

The stores package is built upon the Zustand library and a persistent storage mechanism (persist). This package is responsible for managing the shared application state, particularly user authentication data, JWT tokens and profile data corresponding to each role. Utilizing a centralized state store enables disparate pages to access and update user data consistently without transmitting data through multiple component levels.

The lib package contains backend communication components, wherein api.ts serves as an intermediary layer to transmit RESTful API requests and handle responses from the server. The types and utils packages play a supporting role, providing shared data type definitions and utility functions for data processing, formatting and display. Although these two packages do not contain core business logic, they enhance reusability and improve the maintainability of the frontend source code.

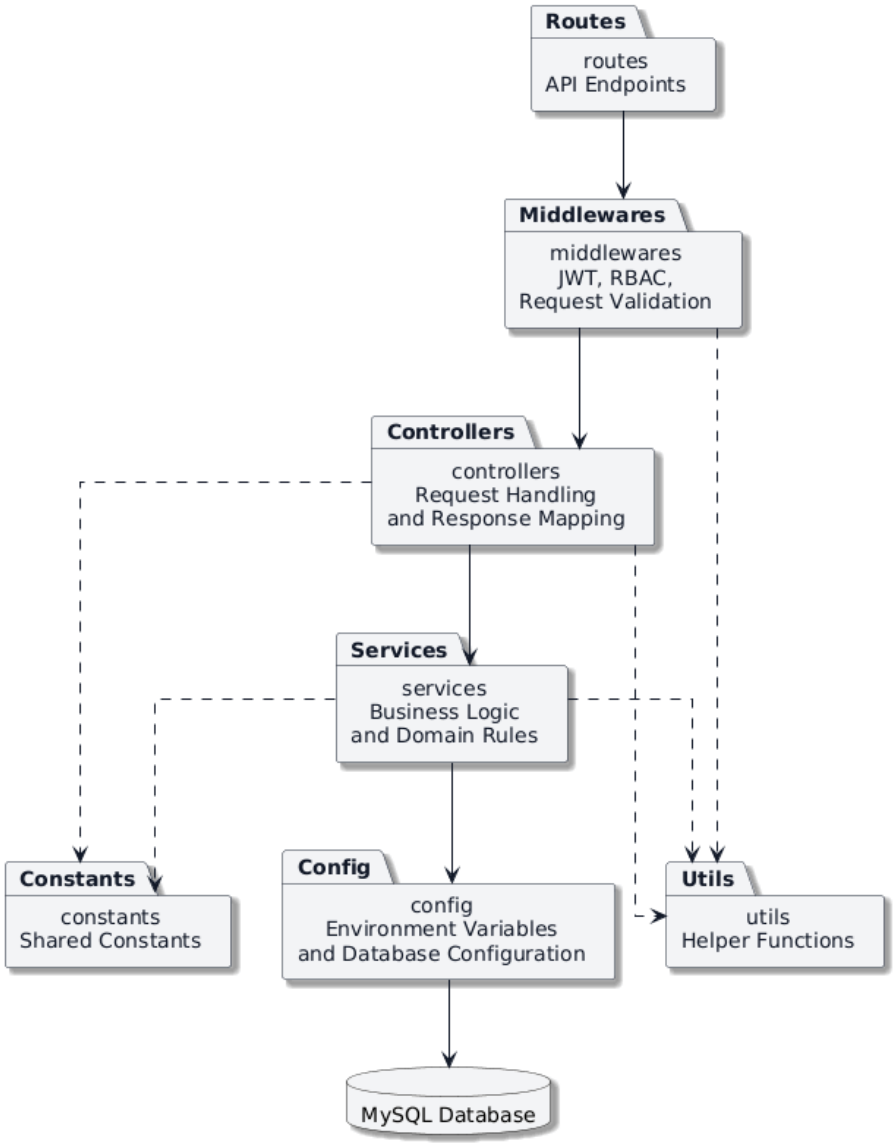


Figure 4.3: Backend Package Diagram

Figure 4.3 describes the package diagram of the backend. The backend is organized into multiple packages following the layered model, where each package assumes a distinct responsibility and communicates solely with adjacent packages.

The routes package defines the system endpoints and maps each HTTP request to its corresponding handling component. Routes do not directly execute business logic; instead, they function exclusively to coordinate and forward requests to the underlying processing layers.

The middlewares package contains cross-cutting processing components of the system, such as JWT authentication, role-based authorization and input data validation. This package is positioned between routes and controllers to ensure that every request undergoes access control verification before entering the business processing logic.

The controllers package is responsible for receiving requests, extracting data from the request body, parameters, or query strings, invoking the corresponding services and standardizing the response data sent back to the frontend. Controllers do not contain complex business logic but primarily serve to coordinate and map data between the API layer and the business logic layer.

The services package is the central component of the backend, where the system business rules are implemented. Business operations such as class configuration, enrollment processing, teaching activity management, monthly tuition fee generation, notification management and constraint verification under Circular No. 29/2024/TT-BGDĐT are all constructed within this package. Centralizing business logic inside services increases source code reusability and minimizes inter-layer dependencies within the system.

The config package is responsible for managing shared application configurations, such as environment variables, database connection configurations and system initialization parameters. This package represents the layer closest to the MySQL database and provides data access mechanisms for the services.

The constants and utils packages are shared components utilized by various elements across the system. The constants package stores fixed constants and configurations utilized throughout the application, whereas the utils package provides utility functions to support data processing and shared tasks. These two packages are designed independently of the main processing flow and do not depend on other business packages.

Both package diagrams indicate that the entire EduCenter system is organized according to the principle of unidirectional dependency from upper to lower layers. Packages in lower layers do not possess reverse dependencies on upper-layer packages and no circular dependencies exist between packages. This organizational approach minimizes coupling between components, heightens system scalability and establishes a favorable foundation for incorporating new functionalities in the future.

4.1.3 Detailed Package Design

Following the presentation of the overall architecture and the package organization of the system, this section describes the detailed design of several representative package groups. Rather than presenting the entire source code at a detailed level, this report selects two critical functional groups: enrollment management and tuition management. These two functionalities represent most components within the system backend architecture, including API definitions, authentication and au-

thorization, business logic processing, data access and notification generation.

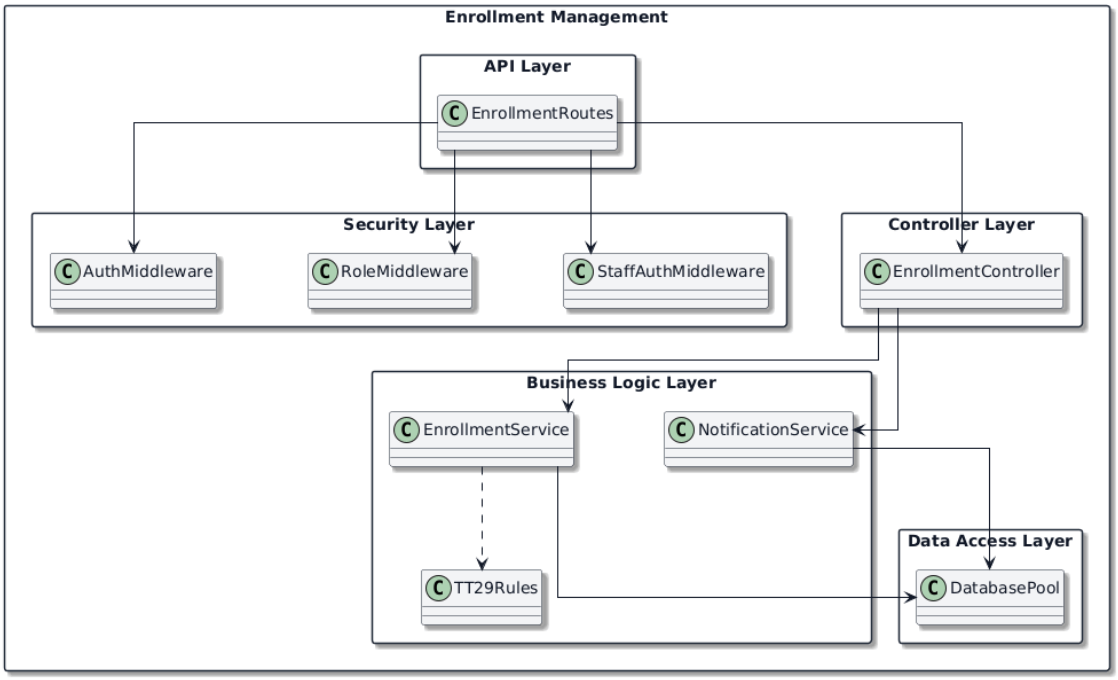


Figure 4.4: Detailed Package Design for Enrollment Management

Figure 4.4 describes the detailed design of the enrollment management package group. Requests associated with enrollment are defined within the EnrollmentRoutes class, which maps RESTful API endpoints to their corresponding handling components. Before a request is forwarded to the business logic layer, the system sequentially executes user authentication via AuthMiddleware, verifies user roles using RoleMiddleware and inspects the staff management scope through StaffAuthMiddleware. Isolating these middlewares prevents controllers from repeating access control verification code and enhances system reusability.

Upon passing the verification steps, the request is forwarded to the EnrollmentController. This class is responsible for receiving request data, verifying the validity of input parameters and coordinating requests to the corresponding services. Core business rules are implemented within the EnrollmentService, encompassing class status verification, current student count inspection, enrollment status processing and the application of operational constraints constructed based on Circular No. 29/2024/TT-BGDĐT. Several shared rules are segregated into the TT29Rules class to minimize source code duplication and enhance maintainability when business regulations change. Once the enrollment status is updated, the NotificationService is utilized to generate and transmit notifications to relevant parties. The services access the database via the DatabasePool, the component responsible for managing connections and executing parameterized SQL queries to the MySQL

database.

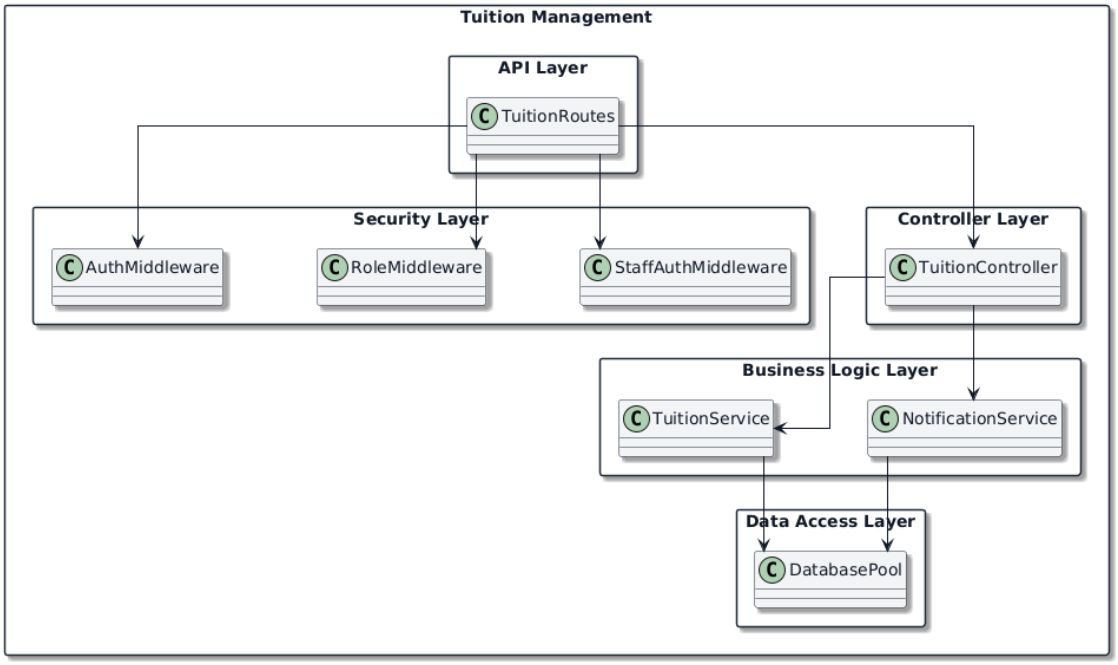


Figure 4.5: Detailed Package Design for Tuition Management

Figure 4.5 describes the detailed design of the tuition management package group. Similar to the enrollment management functionality, endpoints related to tuition are defined inside TuitionRoutes and protected by authentication and authorization middlewares prior to being forwarded to the TuitionController. The controller is responsible for receiving requests, standardizing input data and transferring requests to the business logic layer.

The primary processing is implemented within the TuitionService. This class executes business operations such as generating monthly tuition records, calculating the number of class sessions required for fee collection, determining the total amount payable, updating payment statuses and handling tuition adjustments when necessary. When tuition records are generated or payment statuses change, the NotificationService is employed to transmit notifications to students and related parties. Consistent with other functionalities in the system, tuition data is stored and retrieved via the DatabasePool.

Both detailed design diagrams demonstrate that the critical functional groups of EduCenter comply with the same architectural principles. The processing flow invariably follows a unidirectional path from the API definition layer, through the authentication and authorization layer, to the request coordination layer, followed by the business logic layer and finally to the data access layer. In the current design, the system has not isolated a separate, independent repository package; instead, it utilizes a MySQL connection pool combined with parameterized SQL queries to

execute operations with the database. This approach maintains the separation of responsibilities between layers within the system architecture while keeping the backend structure simple and appropriate for the scope of the thesis.

4.2 Detailed design

4.2.1 User Interface Design

EduCenter is developed as a web application and designed to operate on both desktop and mobile devices. However, since the system frequently displays large volumes of data such as class lists, tuition information and attendance tables, the interface is optimized primarily for desktop and laptop screens. On mobile devices, the system maintains usability through a responsive mechanism, wherein interface components are automatically rearranged to fit smaller display environments. The system utilizes a modern graphical user interface with a semantic color palette to enhance information cognition. Blue is employed as the primary color for navigation elements, core actions and interactive objects, reflecting reliability and professionalism. Green represents successful or completed states. Yellow denotes pending or attention-required states, whereas red is reserved for error, overdue, or canceled states. Utilizing consistent colors enables users to rapidly identify data statuses without reading the entire detailed content. The EduCenter interface is constructed following a dashboard model, in which functional components are divided into distinct zones such as navigation bars, user information areas, statistical cards and the main workspace. This organizational structure reduces interface complexity, heightens navigability and establishes uniformity across functionalities as well as between different user groups.

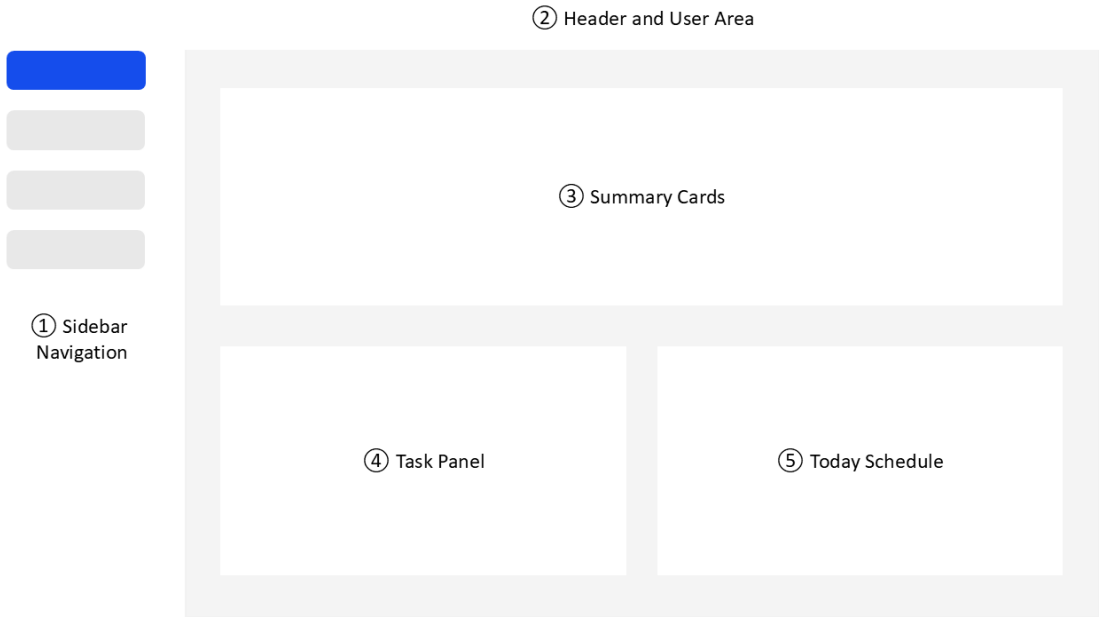


Figure 4.6: Overall User Interface Layout of EduCenter

Figure 4.6 presents the general interface structure that is consistently applied throughout the system. The layout consists of five primary regions, namely the navigation sidebar, the header and user information area, summary information cards, the task panel and the schedule panel. Separating information into independent regions reduces visual complexity and enables users to underst and the current system context immediately after entering the application. This layout also serves as the foundation for designing all functional modules in EduCenter.

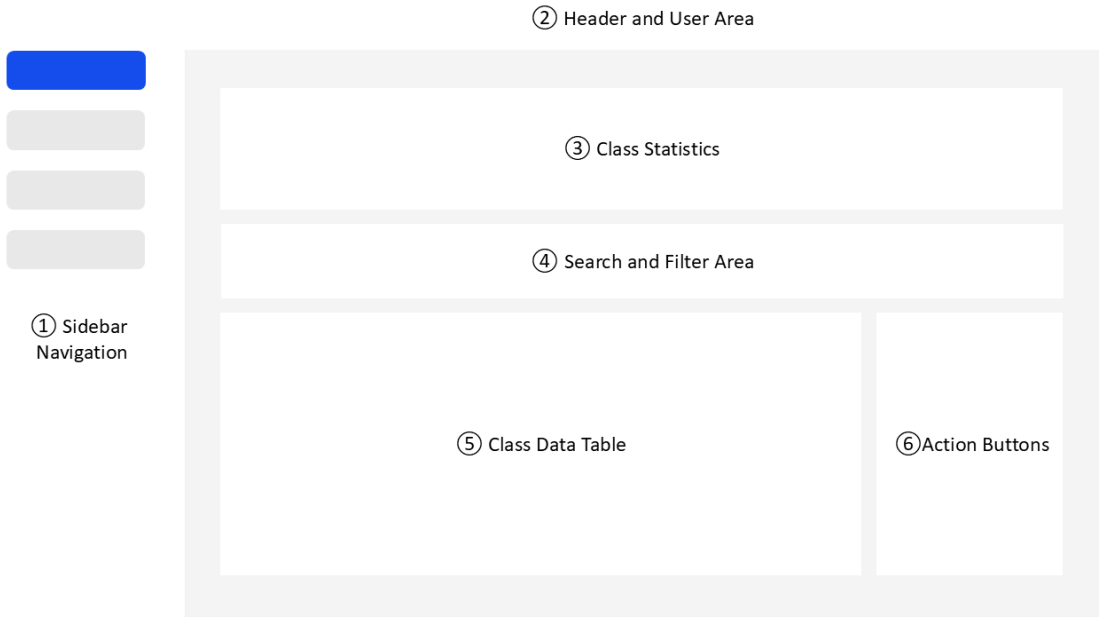


Figure 4.7: Administrative Class Management Interface

Figure 4.7 illustrates the interface used by administrators and staff members to manage classes. The screen combines statistical information, search and filtering mechanisms and a data table within a single workspace. Summary cards provide an overview of the number of classes in different states, while the search and filtering area allows users to narrow down large datasets efficiently. The tabular layout displays detailed information about each class, including teachers, schedules, tuition fees and current statuses. Action buttons are positioned on the right side of each record to support common operations such as viewing, editing and deleting class information.

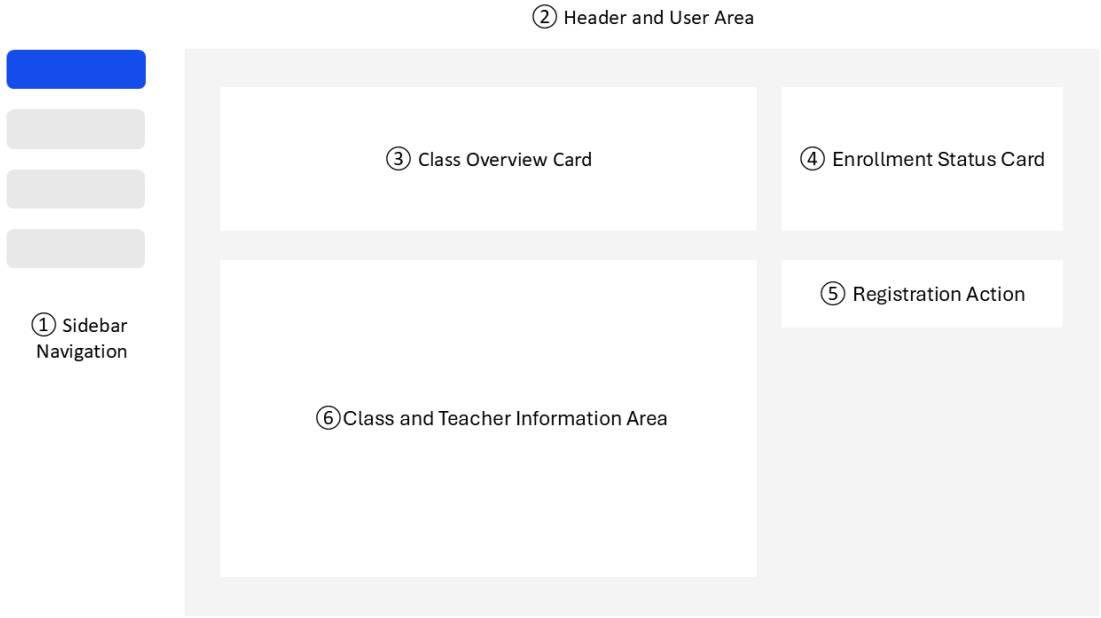


Figure 4.8: Student Class Registration Interface

Figure 4.8 presents the interface designed for students during the class registration process. The screen emphasizes the most important information required for decision making. A large class overview card displays the class name, subject and tuition fee, while a dedicated status card informs students about registration availability and the current number of enrolled students. The registration action is visually separated from informational components and highlighted using a prominent call-to-action button. Additional information regarding schedules, classrooms and teachers is organized into an independent section to improve readability and reduce information overload.

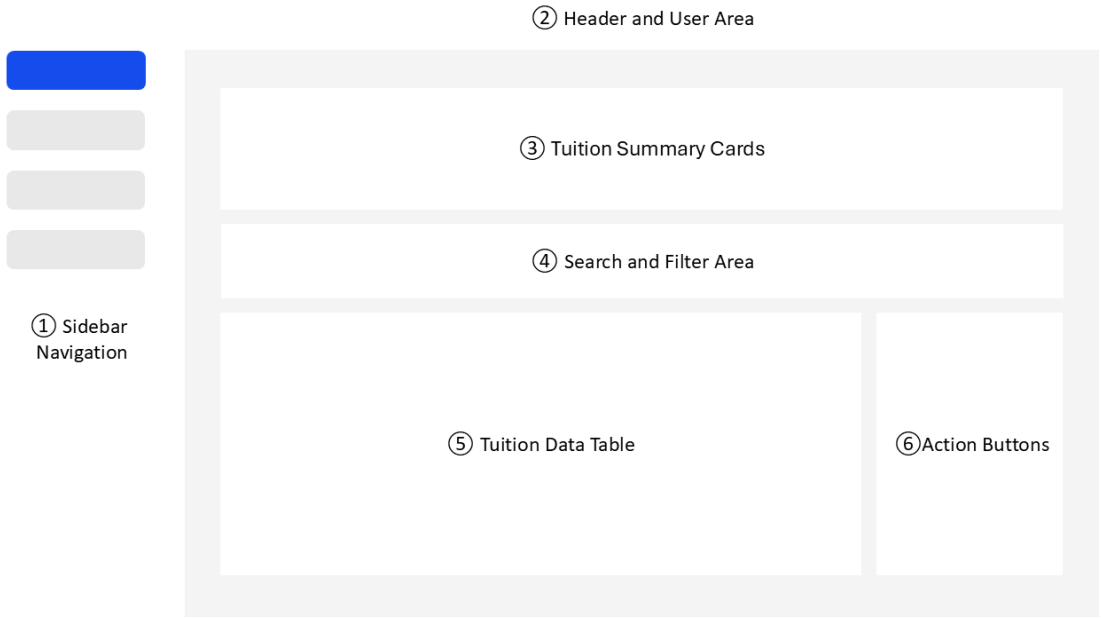


Figure 4.9: Tuition Management Interface

Figure 4.9 illustrates the interface used to monitor and manage tuition information. The screen begins with summary cards that present overall financial indicators, including total tuition, collected payments, outstanding debts and collection rates. Search and filtering components allow users to locate records based on classes, payment periods and payment statuses. Tuition information is then displayed in a tabular form to facilitate comparison and monitoring. Status badges and color coding provide immediate visual feedback regarding payment conditions and help administrators identify unpaid records quickly.

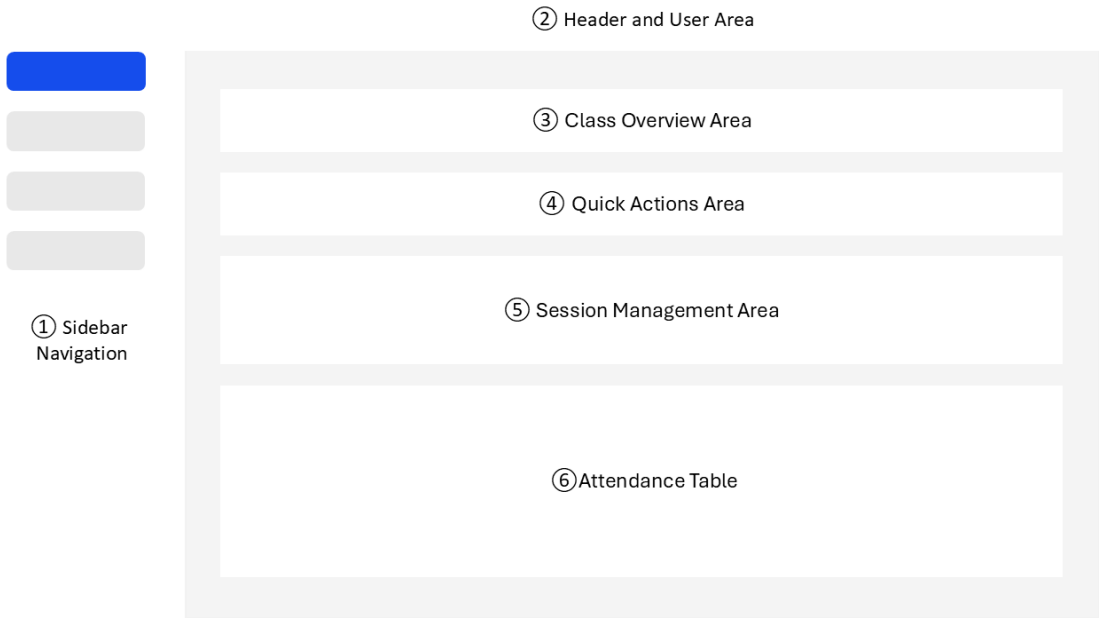


Figure 4.10: Teacher Class and Attendance Management Interface

Figure 4.10 presents the workspace designed for teachers. The interface integrates several closely related activities into a single screen, including class overview information, quick actions, session management and attendance tracking. The class overview section displays essential teaching information such as schedules, classrooms and the number of enrolled students. Frequently used operations, including creating learning sessions and sending notifications, are provided as quick actions. Session management components allow teachers to monitor lesson progress, while the attendance table provides both detailed attendance records and aggregated statistics. Integrating these functions within a single interface minimizes navigation effort and supports teachers in performing routine teaching activities efficiently.

4.2.2 Layer Design

The Student Enrollment Management with Circular 29 Validation functionality was selected to present the detailed class design as it represents one of the core business operations of the system. This functionality simultaneously involves

enrollment management, class condition verification, enforcement of regulations under Circular No. 29/2024/TT-BGDĐT and notification delivery to students and parents. Figure 4.xx presents the class diagram of the primary objects participating in this functionality.

The EnrollmentService class functions as the central business control layer for enrollment management. This class utilizes the pool attribute to interact with the database and provides methods to handle the entire lifecycle of an enrollment request. The create(), approve(), reject()and cancel() methods respectively execute the creation, approval, rejection and cancellation of enrollments. In addition, the class provides the validateThongTu29() method to cross-reference the school and public school class information of the student with the teaching profile of the teacher, thereby detecting violations of regulations under Circular No. 29/2024/TT-BGDĐT. Centralizing the entire enrollment business logic within EnrollmentService ensures consistent execution of business rules and reduces dependencies between system components. The Enrollment class is an entity class representing the enrollment relationship between a student and a class. Each object of this class corresponds to a record in the enrollments table of the database. The class comprises identifier attributes for the student and the class, the registration timestamp, approver information, approval timestamp and the rejection reason. The enrollment status is managed via the EnrollmentStatus enumeration, which consists of four states: PENDING, APPROVED, REJECTED and CANCELLED. This design enables the system to track the complete lifecycle of an enrollment request while simplifying the control of state transitions during business logic processing. The NotificationService class is responsible for generating and managing notifications that arise during the enrollment process. The class utilizes the pool attribute to execute notification data storage and retrieval operations, while providing methods to generate notifications when a student registers successfully, or when a request is approved, rejected, or canceled. Segregating the notification mechanism into an independent class ensures that the enrollment business logic does not depend directly on the implementation details of the notification functionality, thereby increasing the scalability and maintainability of the system.

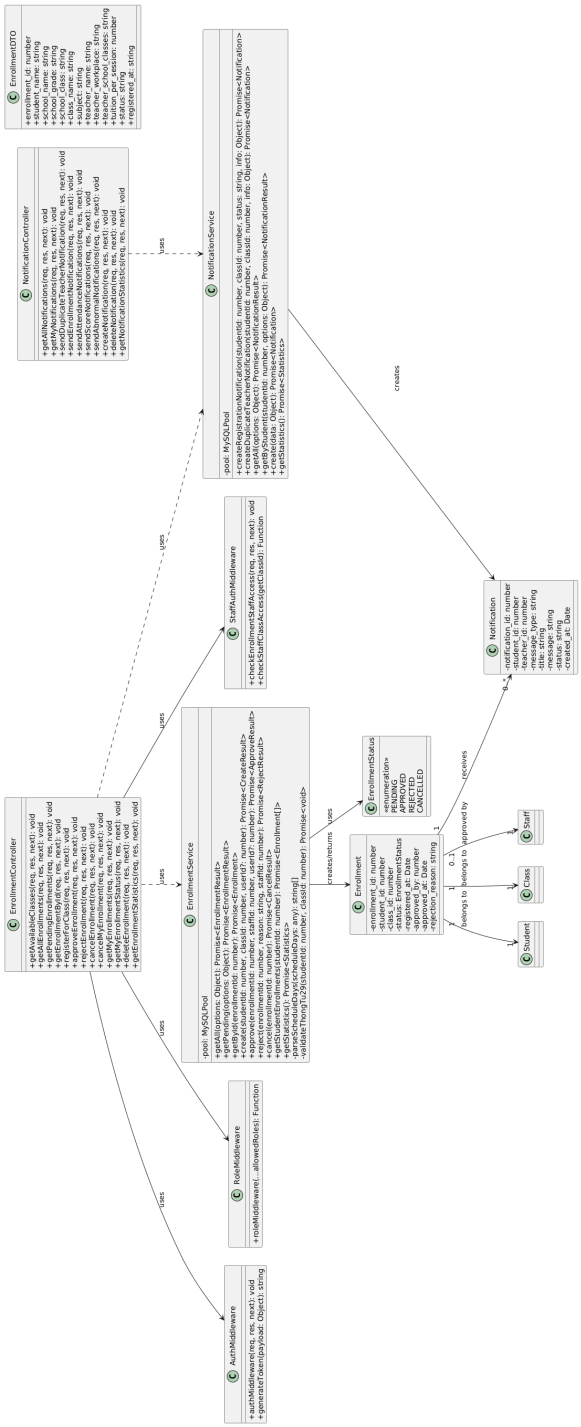


Figure 4.11: Class Diagram for Student Enrollment Management with Circular 29 Validation

The class diagram in Figure 4.11 demonstrates that EnrollmentService acts as the central coordinator for enrollment management functionality. This class interacts directly with Enrollment to create and update enrollment statuses, while employing the NotificationService to generate corresponding notifications. Conversely, Enrollment maintains relationships with the Student, Class and Staff entities, thereby fully representing the information of an enrollment request within the system.

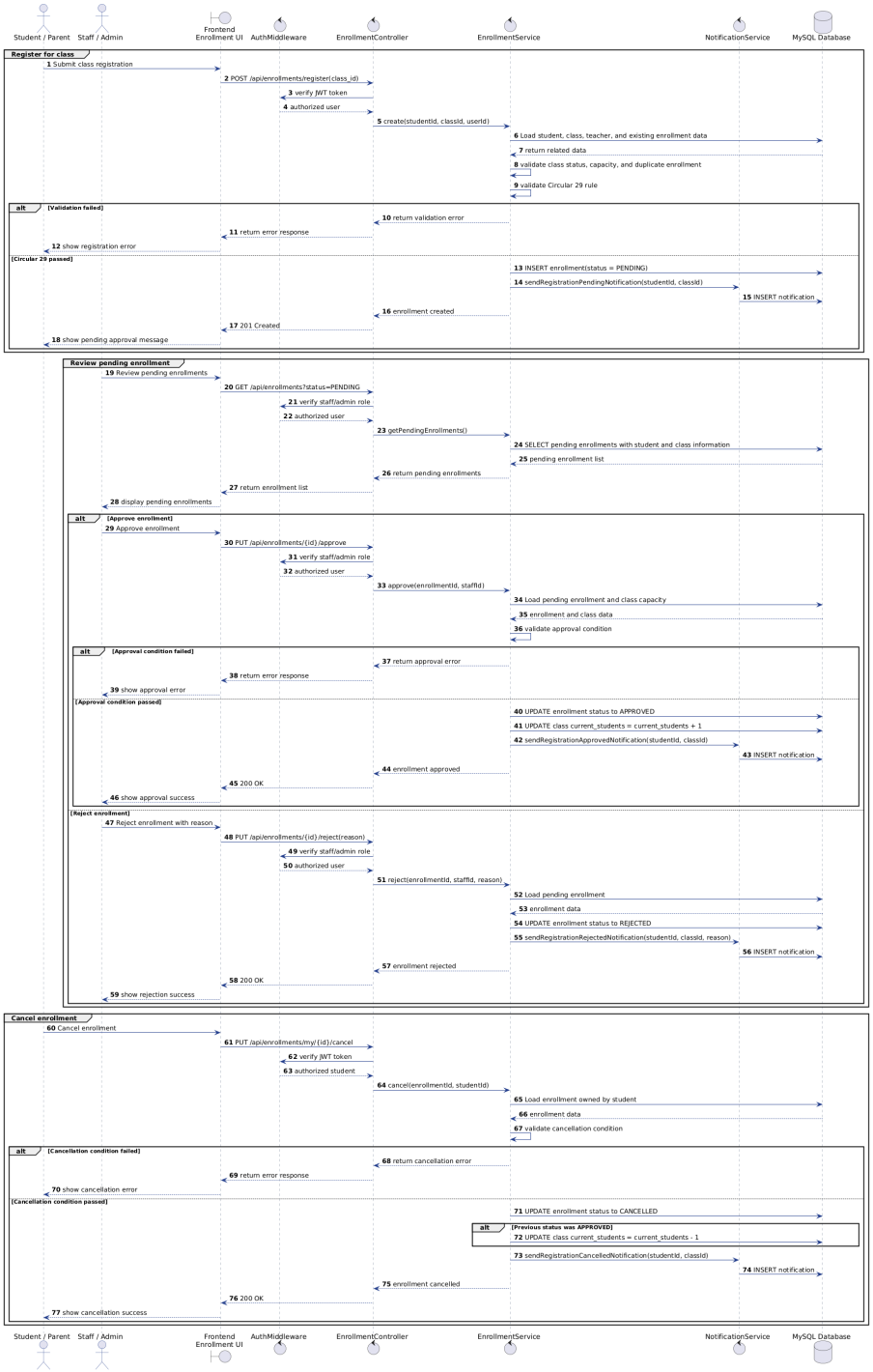


Figure 4.12: Sequence Diagram for Student Enrollment Management with Circular 29 Validation

Figure 4.12 presents the sequence diagram describing the message exchange flow among the objects participating in the student enrollment management functionality. When a student transmits a class registration request, the Enrollment-Controller receives the request and executes authentication via AuthMiddleware prior to forwarding the request to the EnrollmentService. At this layer, the system sequentially verifies the existence of the student, class status, maximum student capacity, existing enrollment status and conditions under Circular No. 29/2024/TT-

BGDĐT. If all conditions are satisfied, a new enrollment record is generated with a PENDING status and the NotificationService creates the corresponding notification associated with the student account. For requests awaiting review, Staff or Admin can execute approval or rejection operations. When a request is approved, the enrollment status is updated to APPROVED and the current student count of the class is incremented accordingly. In the event of a rejection, the enrollment status is updated to REJECTED and the system transmits a notification specifying the rejection reason. The system also supports enrollment cancellation processing by Staff or Admin upon student request. In this scenario, the enrollment status is updated to CANCELLED. If the enrollment had previously been approved, the current student count of the class is adjusted accordingly to maintain data consistency.

4.2.3 Database Design

The EduCenter system uses MySQL as the relational database management system to store and manage application data. A relational database was selected because the system contains highly structured information with clear relationships among users, classes, enrollments, teaching activities and tuition records. Primary keys and foreign keys are used throughout the database to maintain data consistency and support reliable transaction processing.

The database was designed based on the entity-relationship (E-R) model before implementation. The resulting schema consists of thirteen relational tables that collectively support the major functional modules of the system, including user management, class management, enrollment processing, teaching activities, tuition management, notifications and audit logging. Figure 4.13 illustrates the entity-relationship diagram of the EduCenter database.

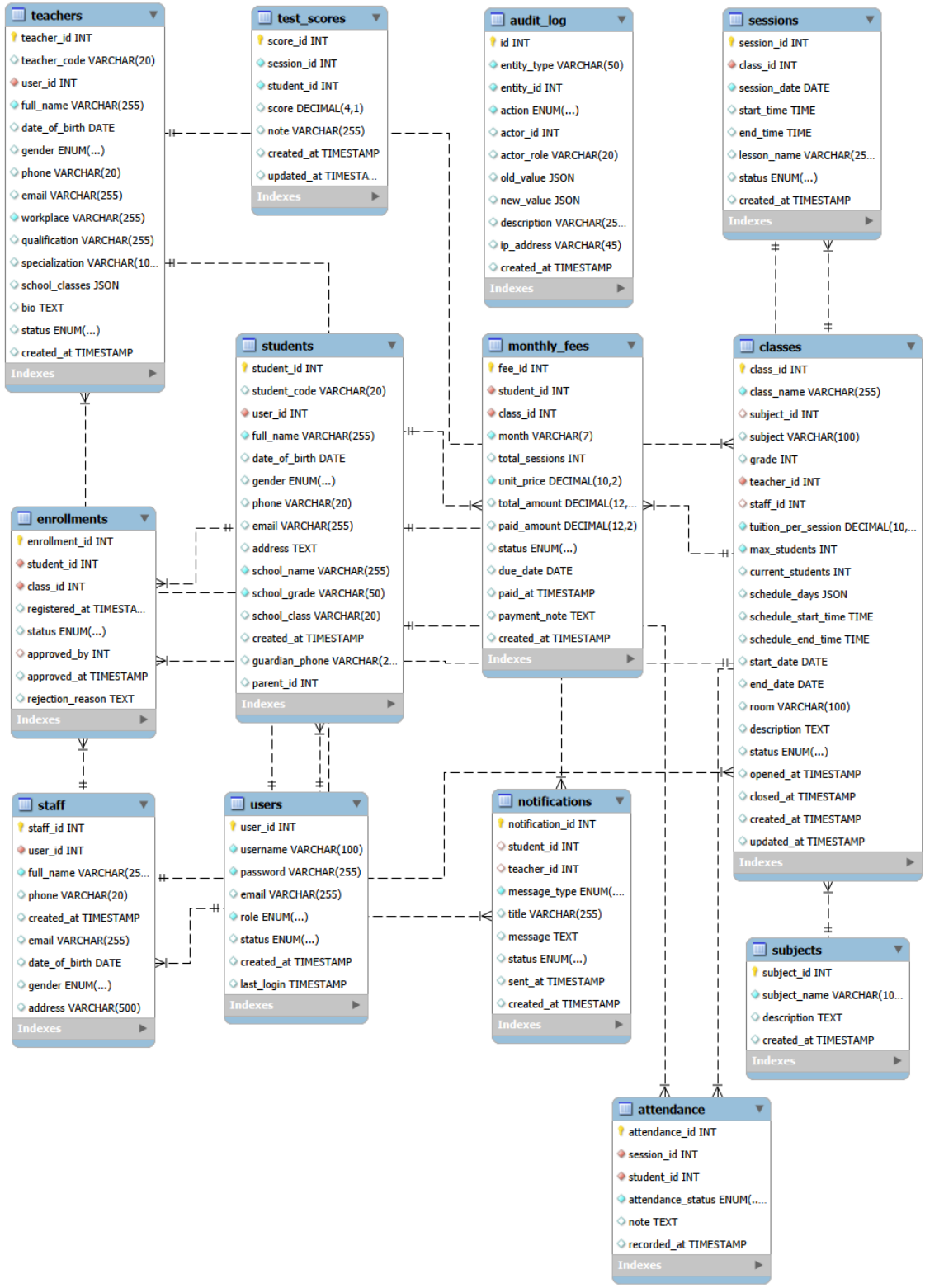


Figure 4.13: Entity-Relationship Diagram of the EduCenter Database

As shown in Figure 4.13, the database can be divided into four groups of entities. The first group consists of the *users*, *students*, *teachers* and *staff* tables, which support authentication and user profile management. The second group includes the *subjects*, *classes* and *sessions* tables for organizing classes and teaching schedules. The third group comprises the *enrollments*, *attendance* and *test_scores* tables,

which manage the student learning process from registration to academic performance. The final group contains the *monthly_fees*, *notifications* and *audit_log* tables, supporting tuition management, communication and system auditing.

Among the thirteen entities shown in Figure 4.13, four tables were selected for detailed description because they represent the core data structure of the EduCenter system, including user management, student information, class organization and enrollment processing.

Attribute	**Data Type**	**Description**
user_id	INT	User identifier (Primary key).
username	VARCHAR(100)	Username used for authentication.
password	VARCHAR(255)	Encrypted password.
email	VARCHAR(255)	User email address.
role	ENUM	User role (Administrator, Staff, Teacher or Student).
status	ENUM	Current account status.
created_at	TIMESTAMP	Account creation time.
last_login	TIMESTAMP	Most recent login time.

Table 4.1: Description of the *users* table

The *users* table, presented in Table 4.1, stores authentication information for all accounts in the system. Each account is associated with a corresponding profile according to its role, providing a unified authentication mechanism while separating role-specific information.

Attribute	**Data Type**	**Description**
student_id	INT	Student identifier (Primary key).
student_code	VARCHAR(20)	Unique student code.
user_id	INT	Reference to the corresponding user account (Foreign key).
full_name	VARCHAR(255)	Student's full name.
date_of_birth	DATE	Student's date of birth.
gender	ENUM	Student gender.

Attribute	Data Type	Description
phone	VARCHAR(20)	Student's phone number.
email	VARCHAR(255)	Student's email address.
address	TEXT	Student's residential address.
school_name	VARCHAR(255)	Current school of the student.
school_grade	VARCHAR(50)	Current grade level.
school_class	VARCHAR(20)	Current school class.
created_at	TIMESTAMP	Record creation time.
guardian_phone	VARCHAR(20)	Parent or guardian contact number.
parent_id	INT	Reference to the parent information (Foreign key).

Table 4.2: Description of the *students* table

Table 4.2 describes the *students* table, which stores personal information and academic information of students. Besides supporting profile management, the school-related attributes are also used during enrollment validation.

Attribute	Data Type	Description
class_id	INT	Class identifier (Primary key).
class_name	VARCHAR(255)	Name of the tutoring class.
subject_id	INT	Reference to the teaching subject (Foreign key).
subject	VARCHAR(100)	Subject name of the class.
grade	INT	Target grade level of the class.
teacher_id	INT	Reference to the assigned teacher (Foreign key).
staff_id	INT	Reference to the assigned staff member (Foreign key).
tuition_per_session	DECIMAL(10,2)	Tuition fee charged for each class session.
max_students	INT	Maximum number of students allowed in the class.

Attribute	Data Type	Description
current_students	INT	Current number of enrolled students.
schedule_days	JSON	Study days of the class within a week.
schedule_start_time	TIME	Starting time of each class session.
schedule_end_time	TIME	Ending time of each class session.
start_date	DATE	Starting date of the class.
end_date	DATE	Ending date of the class.
room	VARCHAR(100)	Classroom or location where the class is organized.
description	TEXT	Additional description of the class.
status	ENUM	Current lifecycle status of the class.
opened_at	TIMESTAMP	Time when the class is opened for registration.
closed_at	TIMESTAMP	Time when the class is closed.
created_at	TIMESTAMP	Record creation time.
updated_at	TIMESTAMP	Most recent update time.

Table 4.3: Description of the *classes* table

Among all entities, the *classes* table shown in Table 4.3 is the central entity of the database. It maintains the basic information of each tutoring class and establishes relationships with subjects, teachers, enrollments and teaching sessions.

Attribute	Data Type	Description
enrollment_id	INT	Enrollment identifier (Primary key).
student_id	INT	Reference to the registered student (Foreign key).
class_id	INT	Reference to the selected class (Foreign key).
registered_at	TIMESTAMP	Registration time.
status	ENUM	Enrollment status (Pending, Approved, Rejected or Cancelled).

Attribute	Data Type	Description
approved_by	INT	Reference to the staff member or administrator approving the request (Foreign key).
approved_at	TIMESTAMP	Approval time.
rejection_reason	TEXT	Reason recorded when a request is rejected.

Table 4.4: Description of the *enrollments* table

The *enrollments* table, presented in Table 4.4, manages the relationship between students and classes. It records the registration history and approval results, allowing administrators and staff members to monitor the enrollment process.

4.3 Application Building

4.3.1 Libraries and Tools

During the development of the graduation thesis, modern tools and libraries were utilized to support the building, testing and source code management of the application. Table 4.5 presents the primary technologies used in the project.

Purpose	Tool	Version	URL Address
Programming IDE	Visual Studio Code	1.10x	`https://code.visualstudio.com`
Frontend framework	Next.js	16.2.6	`https://nextjs.org`
UI library	React	19.2.4	`https://react.dev`
Programming language	TypeScript	5.x	`https://www.typescriptlang.org`
CSS framework	Tailwind CSS	4.x	`https://tailwindcss.com`
State management	Zustand	5.0.13	`https://Zustand.docs.pmnd.rs`
Backend framework	Express	5.2.1	`https://expressjs.com`

Purpose	Tool	Version	URL Address
Database management system	MySQL	8.x	https://www.mysql.com
API testing tool	Postman	11.x	https://www.postman.com
Source code management	Git and GitHub	2.x / Cloud Service	https://git-scm.com

Table 4.5: List of libraries and tools used in EduCenter

Table 4.5 outlines the tools and libraries utilized throughout the system development process. The application is built upon a foundation of Next.js, React and TypeScript on the frontend, combined with Express on the backend and MySQL serving as the database management system. Additionally, Visual Studio Code, Postman, Git and GitHub were employed to facilitate development, testing and system source code management.

4.3.2 Achievement

Following the completion of the analysis, design and implementation phases, the EduCenter tutoring center management system was successfully developed as a fully functional web application. Within the scope of this graduation thesis, all planned core modules were completed and integrated into a unified platform that supports the daily operation of a tutoring center. The implemented system covers the principal business processes identified during the requirement analysis phase, including account management, class configuration, student enrollment, teaching activity monitoring, attendance recording, tuition management, notification delivery and operational reporting. In addition to supporting routine administrative activities, the system incorporates selected business rules derived from Circular No. 29/2024/TT-BGDĐT, enabling the automatic validation of enrollment requests and class configurations in accordance with current educational regulations.

The completed product consists of three integrated software components. The frontend application, developed using Next.js and React, provides responsive user interfaces for administrators, staff members, teachers and students across desktop and mobile devices. The backend application, implemented with Node.js and Express.js following a RESTful API architecture, is responsible for authentication, business logic execution and data processing. All operational data are stored and

managed in a MySQL relational database, which maintains data consistency while supporting efficient querying, updating and transaction management. Together, these three components form a complete deployment package that can be installed and operated as a unified tutoring center management system.

The completed implementation demonstrates that the proposed architecture and technology stack successfully satisfied the functional and non-functional requirements defined for the project. Rather than supporting isolated management tasks, the resulting system integrates previously fragmented operational workflows into a single platform with consistent data management, role-based access control and automated business rule enforcement. This outcome reflects the primary objective of the graduation thesis, namely the development of a practical software solution capable of supporting the digital transformation of tutoring center administration.

Table 4.6 summarizes the general information of the completed EduCenter system, including its deployment components and principal implementation technologies. Table 4.7 presents statistical information regarding the implementation scale of the completed application. These statistics provide an overview of the source code size, database structure, API implementation and functional coverage achieved within the scope of this graduation thesis.

Item	**Value**
Application Type	Web Application
Frontend Framework	Next.js 16 + React 19
Backend Framework	Express.js + Node.js
Database Management System	MySQL 8.0
Programming Languages	TypeScript, JavaScript
User Roles	4
Deployment Components	Frontend Application, Backend API, MySQL Database

Table 4.6: Overall Project Information

The statistics presented in Tables 4.6 and 4.7 demonstrate that the EduCenter system reached a substantial implementation scale. The completed application consists of more than 34,000 lines of source code distributed across 134 source files and provides 157 REST API endpoints supported by 13 relational database tables. Together with 13 implemented use cases, 10 core functional modules and three business workflows, these results indicate that the developed system satisfies the principal operational requirements of a tutoring center and represents a complete software product rather than a prototype.

Metric	Value
Source Files	134
Lines of Code	~34,624
Database Tables	13
REST API Endpoints	157
Implemented Use Cases	13
Core Functional Modules	10
Business Process Workflows	3
Operational Reports	5

Table 4.7: System Implementation Statistics

4.3.3 Illustration of main functions

To illustrate the deployment results, this section presents several representative user interfaces of the EduCenter system. The selected interfaces represent core functionalities and prominent system characteristics, spanning user administration, class configuration, specialized operations such as enrollment under the regulations of Circular No. 29/2024/TT-BGDĐT and tuition management.

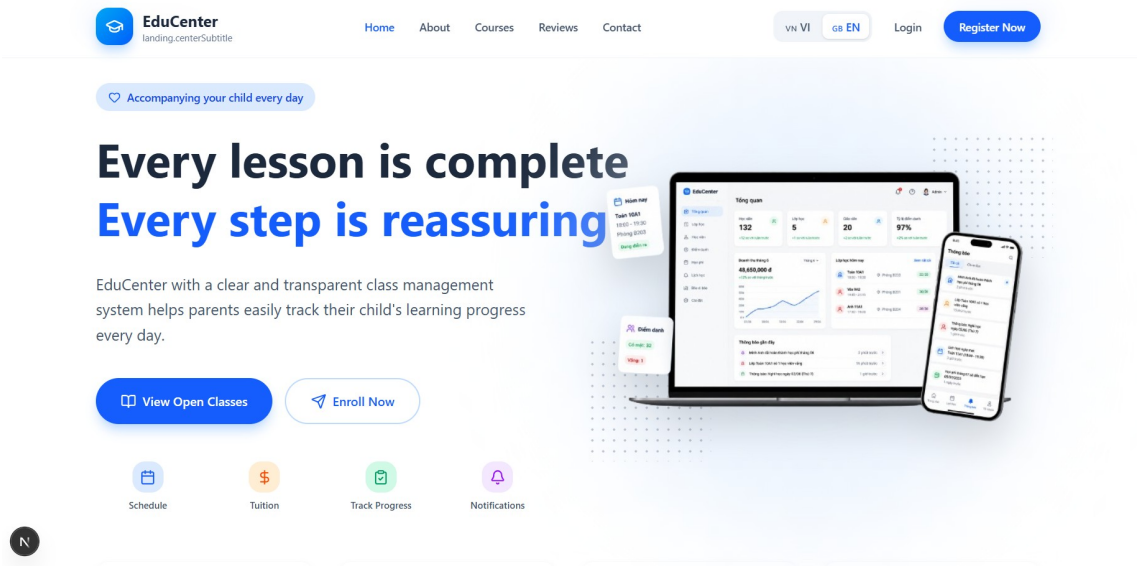


Figure 4.14: Landing Page Interface

The landing page serves as the public entry point of the EduCenter system. It introduces the main objectives of the platform, highlights key features and provides navigation to authentication functions. The page is designed to present essential information about the tutoring center in a clear and accessible manner while allowing prospective students to explore available services before accessing the system.

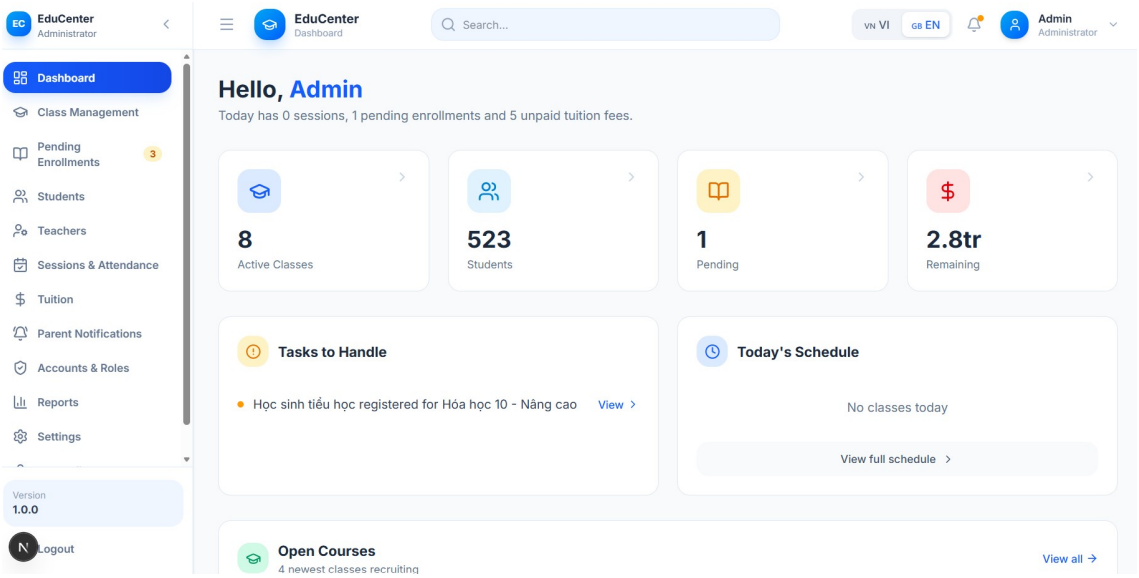


Figure 4.15: Dashboard Interface

The dashboard interface provides a holistic view of center operations through statistical data, such as the number of active classes, student counts, teacher counts and pending requests requiring processing. These metrics are displayed using statistical cards and task lists, enabling users to rapidly grasp the operational state of the center.

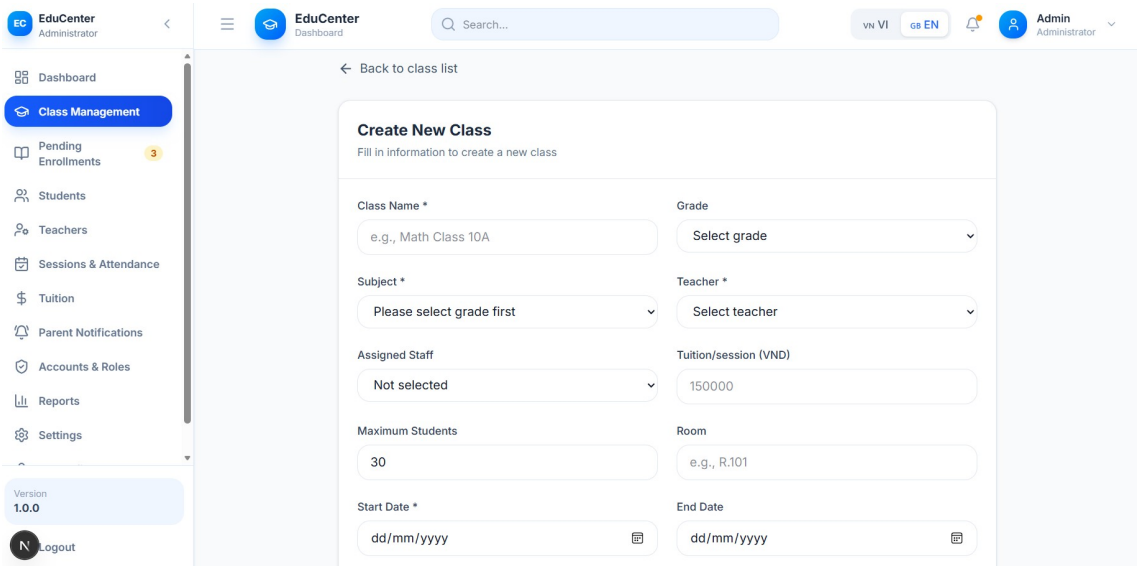


Figure 4.16: Class configuration Interface

This interface allows administrators to set up and manage the classes of the center, encompassing class information, subjects, schedules, assigned teachers, tuition rates and maximum student capacities. The system supports automated session creation based on the configured schedule while validating business constraints before any class transitions to an active status.

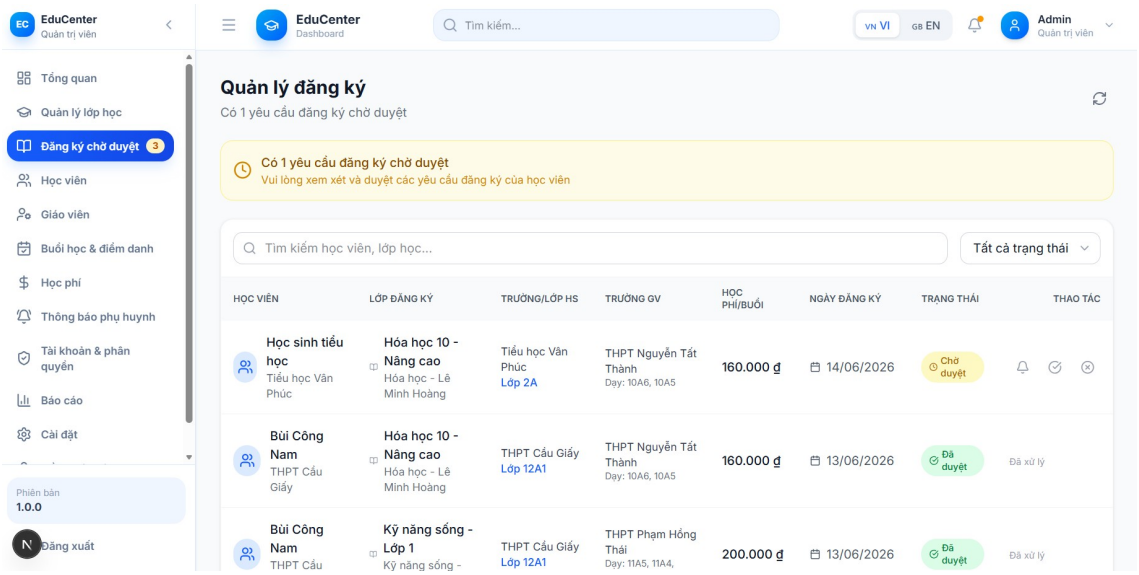


Figure 4.17: Enrollment management Interface

The enrollment management interface supports staff and administrators in tracking and approving student class registration requests. The screen displays comprehensive details regarding student information, registered classes and the processing status of each request. The system simultaneously performs automated checks against enrollment conditions regulated by Circular No. 29/2024/TT-BGDĐT prior to allowing registration profile approvals.

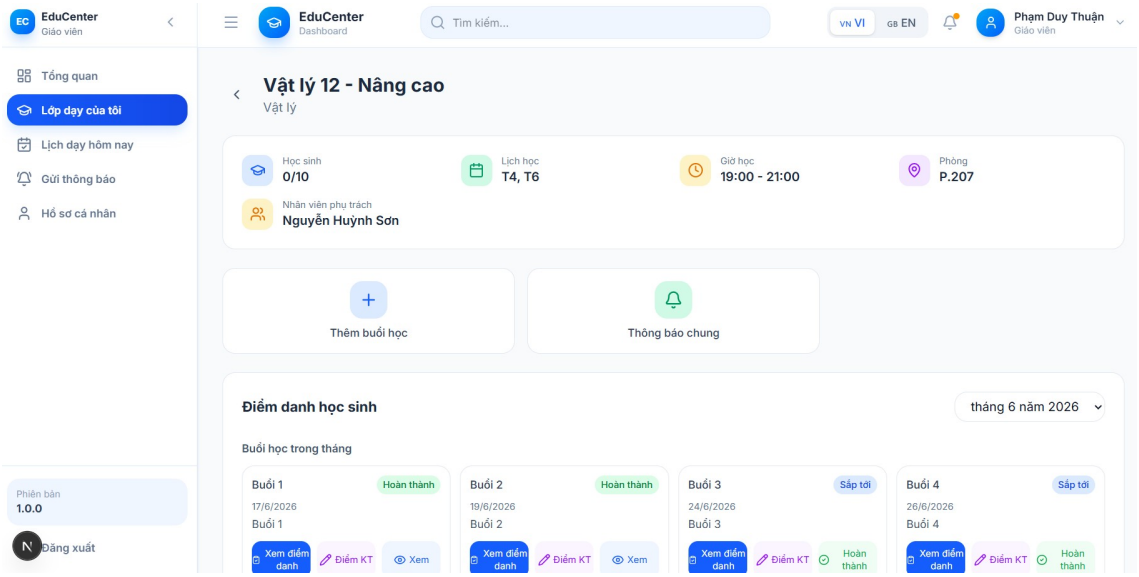


Figure 4.18: Teaching activity tracking Interface

This interface supports teachers in managing their assigned classes, tracking session schedules and executing pedagogical duties. Teachers can create new class sessions, record attendance, update examination results and transmit notifications to students. All data is stored centrally to facilitate academic management and the tracking of student progress.

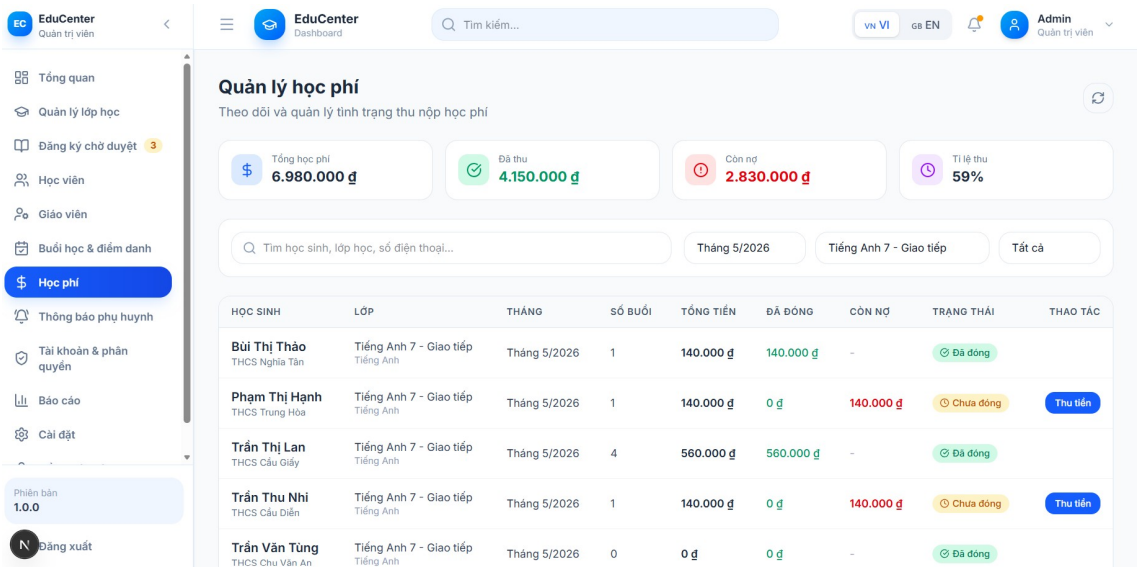


Figure 4.19: Tuition management Interface

The tuition management interface enables the monitoring of student payment statuses organized by specific classes and academic months. The system supports tuition calculation based on actual session attendance, logs completed payments, identifies outstanding balances and displays aggregated indices concerning the fee collection status of the center.

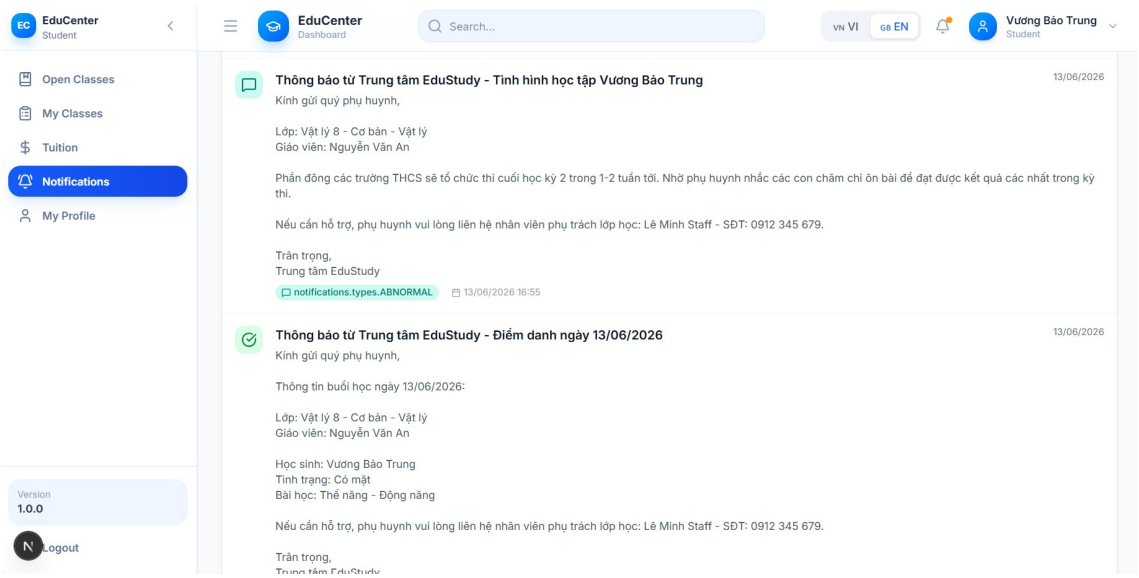


Figure 4.20: Notification Interface

The notification interface allows users to monitor alerts relevant to center activities, including notifications regarding class registration, academic performance, schedules and other administrative updates. Centralizing all notifications into a single unified interface enhances communication capabilities between the center, teachers and students.

4.4 Testing

In the system testing phase, a black-box testing methodology combined with manual testing was applied to evaluate the primary functions of the EduCenter system. Test cases were constructed based on functional requirements, input data constraints and system business rules. During the design of the test cases, equivalence partitioning, boundary value analysis and business scenario testing techniques were integrated to evaluate the correctness and stability of the software. Specifically, test scenarios were developed for three critical functions including class registration, teaching activity tracking and tuition processing.

No.	**Test Data**	**Test Scenario**	**Expected Status**	**Actual Status**	**Result**
1	Student not logged in	The student accesses the class details page and clicks "Register" without logging in	The system requires the user to log in before registering for the class	The system requires the user to log in before registering for the class	Pass
2	Class open and available slots	The student selects an open class and clicks "Register"	The system creates an enrollment request with a PENDING status	The system creates an enrollment request with a PENDING status	Pass
3	Class capacity reached	The student selects a class that has reached its maximum student capacity	The system notifies that the class is full and rejects the registration	The system notifies that the class is full and rejects the registration	Pass
4	Class not in open state	The student performs registration for a class that is not yet open or has closed	The system does not permit registration for this class	The system does not permit registration for this class	Pass

No.	Test Data	Test Scenario	Expected Status	Actual Status	Result
5	Duplicate class registration	The student registers for a class that has an existing prior enrollment request	The system notifies that the student has already registered for this class	The system notifies that the student has already registered for this class	Pass
6	Primary student registering for academic subject	A primary school student performs registration for a Mathematics class	The system rejects the registration according to the regulations of Circular No. 29/2024/TT-BGDĐT	The system rejects the registration according to the regulations of Circular No. 29/2024/TT-BGDĐT	Pass
7	Student registering for class instructed by their main school teacher	A student performs registration for a class instructed by their own teacher from their main school	The system rejects the registration according to the regulations of Circular No. 29/2024/TT-BGDĐT	The system rejects the registration according to the regulations of Circular No. 29/2024/TT-BGDĐT	Pass

Table 4.8: Test results for the class registration function

Table 4.8 describes the test results for the class registration function. Across the constructed test scenarios, the system precisely processed both valid and invalid registration cases. Crucially, specialized business rules built on the basis of Circular No. 29/2024/TT-BGDĐT were properly executed, helping restrict enrollment instances that do not comply with the regulations.

No.	Test Data	Test Scenario	Expected Status	Actual Status	Result
1	Create new class session	The teacher selects the function to create a class session and inputs comprehensive information	The system successfully creates the class session and displays it in the list	The system successfully creates the class session and displays it in the list	Pass
2	Missing session information	The teacher creates a class session but does not input the academic date or teaching time	The system displays an error message and does not save the data	The system displays an error message and does not save the data	Pass
3	Record student attendance	The teacher performs attendance tracking for students in the class	The system saves the attendance status of each individual student	The system saves the attendance status of each individual student	Pass
4	Update attendance status	The teacher changes the attendance status of a student from absent to present	The system updates the corresponding attendance status accordingly	The system updates the corresponding attendance status accordingly	Pass
5	Valid grade entry	The teacher inputs a grade within the permitted range	The system saves and displays the academic results of the student	The system saves and displays the academic results of the student	Pass

No.	Test Data	Test Scenario	Expected Status	Actual Status	Result
6	Invalid grade entry	The teacher inputs a grade value exceeding the permitted range	The system displays an error message and does not save the data	The system displays an error message and does not save the data	Pass

Table 4.9: Test Results for the Teaching Activity Tracking Function

Table 4.9 presents the test results for the teaching activity tracking function. The results demonstrate that the system can accurately create and manage class sessions, record and update attendance information and store student academic results. Invalid input cases were correctly detected and handled by the system, which provided appropriate feedback messages to users and prevented incorrect data from being saved.

Table 4.10: Test Results for the Tuition Processing Function

No.	Test Data	Test Scenario	Expected Status	Actual Status	Result
1	Generate first month tuition	The manager selects the monthly tuition generation function	The system creates a tuition record with an UNPAID status	The system creates a tuition record with an UNPAID status	Pass
2	Regenerate existing tuition	The manager performs tuition generation for a month that already contains data	The system does not create a duplicate tuition record	The system does not create a duplicate tuition record	Pass

No.	Test Data	Test Scenario	Expected Status	Actual Status	Result
3	Calculate tuition from present sessions	The system calculates tuition fees based on the sessions the student attended	The total number of billable sessions and the total tuition amount are accurately calculated	The total number of billable sessions and the total tuition amount are accurately calculated	Pass
4	Calculate tuition with late attendance	The system calculates tuition when a student has a LATE attendance status	The late session is counted as a valid session for tuition fee calculation	The late session is counted as a valid session for tuition fee calculation	Pass
5	View student tuition	The student accesses the personal tuition viewing function	The system correctly displays the tuition list of the logged-in student	The system correctly displays the tuition list of the logged-in student	Pass
6	Confirm tuition payment	The student confirms that the tuition has been paid	The system updates the tuition status to PAID	The system updates the tuition status to PAID	Pass

Table 4.10 presents the test results for the tuition processing function. The results indicate that the system accurately calculates the number of billable sessions, correctly updates payment statuses and maintains the consistency of tuition records. Furthermore, the functionality demonstrates the successful integration between attendance records, class information and monthly tuition management processes.

Within the scope of this thesis, a total of nineteen test cases were designed and executed for the three core functionalities of the EduCenter system. The testing outcomes show that all test cases produced the expected results, yielding a all test cases produced the expected results. The implemented features operated reliably, data was processed accurately and the specialized business rules were enforced in

accordance with the defined requirements. Since no failed test cases were identified during the testing process, a detailed root-cause analysis is not required in this section. Additional test cases may be included in the appendix if further evidence is needed.

This chapter has presented the architecture design, detailed design, implementation outcomes and testing activities of the EduCenter system. The selected architecture and package structure provided a clear separation of responsibilities among system components, while the database and user interface designs effectively supported the operational requirements identified during the analysis phase. The implementation successfully integrated the core processes of tutoring center management, including enrollment processing, teaching activity monitoring, tuition management, notification handling and the enforcement of business rules derived from Circular No. 29/2024/TT-BGDĐT. The testing outcomes confirmed that the implemented functionalities operated as intended and satisfied the specified requirements. These results provide a foundation for discussing the major technical contributions and key solution highlights of the project in the following chapter.

CHAPTER 5. SOLUTION AND CONTRIBUTION

Chapter 4 presented the architecture, detailed design and implementation results of the EduCenter system. While those sections focused on describing how the system was constructed, they only provided a high-level overview of several important mechanisms embedded within the application. Some of these mechanisms represent key technical contributions of the project because they address practical operational challenges commonly encountered in tutoring center management. This chapter examines three representative solutions developed during the implementation process. The first solution transforms selected requirements from Circular No. 29/2024/TT-BGDĐT into a multi-layer enrollment control mechanism capable of automatically validating regulatory compliance. The second solution introduces an automated tuition generation approach that derives financial records from attendance and learning activities. The third solution focuses on schedule automation through configuration-driven session generation and teaching conflict detection. Together, these solutions demonstrate how business requirements, regulatory constraints and software engineering principles were integrated to improve the reliability and operational effectiveness of the EduCenter system.

5.1 Transforming the Regulations of Circular No. 29/2024/TT-BGDĐT into a Multi-Layer Enrollment Control Mechanism in the EduCenter System

5.1.1 Issue Introduction

Extra-classes and tutoring operations are not merely conventional administrative tasks; they are strictly governed by legal regulations within the educational sector. Circular No. 29/2024/TT-BGDĐT stipulates numerous requirements regarding eligible tutoring participants, prohibited tutoring arrangements, class size limitations and teaching durations. These mandates are presented as legal text in natural language, whereas a software system demands explicit, verifiable and automatically enforceable conditions.

In day-to-day operations, manual verification of these conditions requires center staff to cross-reference student, teacher, class and schedule information prior to accepting any enrollment request. This method may suffice for small-scale centers, but it becomes highly inefficient as the volume of students, teachers and classes expands. A minor oversight during verification can lead to students enrolling in inappropriate classes, teachers being assigned to students from their regular school courses, or classes exceeding the legally prescribed limits.

Many contemporary tutoring management systems focus primarily on data dig-

itization, handling records such as student profiles, classes, tuition fees and attendance tracking. While these systems provide better administrative support compared to manual techniques, they often fail to integrate legal regulations into self-regulating business logic. Consequently, the system remains heavily reliant on the experience and attentiveness of the operator.

To address this issue, EduCenter was designed not just to store data, but to actively validate the legitimacy of business operations. The core of this solution lies in transforming key regulations from Circular No. 29/2024/TT-BGDĐT into executable software business rules, particularly within the class creation, class update and student enrollment workflows.

5.1.2 Solution

The developed solution separates processing into three primary steps: legal regulation analysis, business rule modeling and the integration of these rules into system processing flows. Figure 5.1 illustrates this transformation process. From the text of Circular No. 29/2024/TT-BGDĐT, the system selects clauses that can be verified using existing data, subsequently translating them into specific validation conditions within the business layer. These conditions are reused across multiple distinct operations, including class creation, class updates and enrollment request processing.

An alternative implementation strategy would be to perform all regulatory checks only when a student submits an enrollment request. Although this approach reduces the amount of validation logic executed during class management operations, it allows invalid class configurations to remain in the database until an enrollment attempt occurs. Such a design may lead to inconsistencies between stored class data and regulatory requirements. EduCenter therefore adopts an earlier validation strategy in which compliance checks are distributed across class creation, class modification and enrollment processing stages. This design prevents invalid configurations from propagating through later business workflows and improves overall data integrity.

At the data level, the system stores the necessary information required for validation, encompassing the grade level and regular school class of students, the employment institution and assigned teaching schedules of teachers, subject details, maximum student capacities and the specific timetable of each class. Incorporating these data fields provides the system with a sufficient foundation to automatically cross-reference enrollment requests against business constraints.

For primary school students, the system applies the rule derived from Article

4.1 of the Circular. According to this clause, primary school students are prohibited from attending extra academic classes, with the exception of extracurricular activities involving arts, physical education and life skills training. Within EduCenter, this regulation is implemented by identifying the grade level of the student and validating the subject matter of the registered class. If the student belongs to a primary grade level and the class falls outside the permitted group of subjects, the system rejects the enrollment request.

For teachers, the system applies the rule from Article 4.2 of the Circular. This regulation states that teachers currently employed at public or private schools may not conduct paid extra-classes outside of school for students whom they are officially assigned to teach in their regular curriculum. To validate this condition, EduCenter cross-references the workplace of the teacher with the school of the student, while simultaneously matching the grade level and regular class information of the student against the list of classes taught by the teacher at their main institution. If a match occurs, the enrollment request is rejected before being committed to the system.

Furthermore, EduCenter monitors several class organization conditions outlined in Article 5.4 of the Circular. The system enforces class size limits through the maximum student capacity field and prevents further student approvals once a class is full. Regulations regarding weekly tutoring durations per subject are modeled as constraints on the frequency of scheduled sessions per week for the same subject, teacher and grade level. Hence, the system can detect incompatible class configurations during the initial creation or update phases.

In addition to the rules deduced directly from the Circular, the system includes supplementary business rules to guarantee data consistency. Students cannot register twice for the same class, cannot enroll in classes that are not yet open for registration and are barred from participating in classes with overlapping schedules. These rules do not replace legal regulations, but they ensure tighter operation of the enrollment process and eliminate discrepancy scenarios in practice.

A key design decision of EduCenter is the adoption of a Defense in Depth validation strategy. Instead of concentrating all regulatory verification logic at a single processing point, the system distributes validation responsibilities across multiple stages of the class lifecycle. Figure 5.2 illustrates this architecture. Compliance rules are enforced during class creation, class modification and enrollment processing. Each stage operates as an independent control layer capable of detecting invalid conditions before they propagate further into the system. When an admin-

istrator creates a class, the system checks conditions related to subjects, schedules, capacities and weekly sessions. When class information is updated, these conditions are verified again to prevent initially valid data from being modified into an invalid state. When a student submits an enrollment request, the system executes a final layer of validation prior to generating the registration record.

This organizational structure mitigates risks caused by configuration errors or inaccurate user operations. If a class is incorrectly created or inappropriately updated, the enrollment request can still be intercepted at a later stage. Conversely, if an enrollment request is invalid due to student or teacher information, the system can reject it without affecting existing class data. Therefore, enrollment data within the database is protected by multiple independent control layers rather than depending on a single validation checkpoint.

From a software engineering perspective, this architecture improves maintainability and fault tolerance. Individual validation rules can be modified or extended without redesigning the entire enrollment workflow because each control layer remains logically independent. The approach also reduces the impact of configuration errors. Even if an invalid condition is not detected during an earlier operation, subsequent validation stages can still prevent inconsistent enrollment data from being stored.

5.1.3 Achieved Results

The solution successfully established an enrollment control mechanism within EduCenter capable of automatically detecting and preventing multiple non-compliant or business-invalid scenarios. Instead of relying entirely on manual checks by center personnel, the system automatically cross-references student, teacher, class and schedule information before allowing an enrollment request to be generated.

More importantly, the solution transformed regulatory requirements from static legal documents into executable business constraints. This transformation allows compliance verification to be performed consistently and automatically for every enrollment transaction, removing the dependency on manual interpretation by center personnel.

Transforming legal regulations into business rules highlights the distinct differences between this system and tutoring center software that focuses solely on data storage. EduCenter not only supports users in executing administrative actions but also functions as a business control layer, helping reduce human errors and enhance regulatory compliance within tutoring activities.

The multi-layer validation mechanism also increases overall system reliability. Rules are applied during class creation, class updates and student enrollment, restricting invalid data from entering the database. This approach is highly suited for educational management systems, where data must be correct technically and must also conform to business constraints and administrative regulations.

Another critical outcome of the solution is its maintainability and extensibility. Because the rules are modeled as independent validation conditions, the system can incorporate or adjust rules when regulations change without requiring a complete redesign of the enrollment workflow. This serves as a foundation for EduCenter to evolve into a tutoring center management system capable of adapting to shifting legal environments and practical operational demands.

5.2 Automated Tuition Generation Solution Based on Actual Attendance Data

5.2.1 Issue Introduction

In tutoring center operations, tuition fees are typically determined based on the number of class sessions that students actually attend each month. Many contemporary management systems still apply manual tuition calculation methods or calculate fees according to the projected number of class sessions. This approach is relatively straightforward, yet it easily introduces discrepancies when class schedules change, teachers update additional sessions, or attendance tracking is finalized after the tuition records have already been generated.

Manual calculation also increases the workload of administrative personnel significantly. To generate tuition for a class, the person in charge must simultaneously cross-reference the list of enrolled students, the status of class sessions and the corresponding attendance data. As the number of classes and students increases, data aggregation becomes complex and carries a potential risk of errors. An inaccurate tuition record can diminish system transparency and create difficulties during reconciliation between the center, students and parents.

Driven by this operational reality, EduCenter was designed to treat tuition as a derived result of actual academic data rather than as data entered directly by users. The objective of this solution is to automatically aggregate data from multiple distinct sources to precisely determine the number of billable sessions and the corresponding tuition amount for each student every month.

An alternative design would allow administrative staff to manually enter tuition amounts for each student. Although this approach is simpler to implement, it introduces a significant risk of inconsistency because tuition data becomes detached from actual learning activities. Any later modification to attendance records or

class sessions would require manual recalculation. EduCenter therefore treats tuition as derived data generated from enrollment, session and attendance information, ensuring that financial records remain synchronized with academic records.

5.2.2 Solution

The solution is constructed on the principle of using only data that has been confirmed during the learning process. Figure 5.3 illustrates the tuition generation workflow within the system. The calculation process begins with the list of students whose class enrollment has been approved. For each student, the system collects class sessions with a completed status in the corresponding month, subsequently cross-referencing this with attendance data to determine the number of billable sessions.

Within EduCenter, only class sessions with a completed status are included in the tuition calculation process. Simultaneously, only attendance records marked with full participation or late attendance are regarded as valid sessions for fee calculation. Absenteeism or excused absences are not added to the number of tuition-billable sessions. From this data, the system determines the billable sessions of each individual student for the month.

Following the determination of valid sessions, the system applies the formula:

$$\text{Total Tuition} = \text{Billable Sessions} \times \text{Tuition per Class Session} \tag{5.1}$$

This approach reflects an important design principle adopted in EduCenter: tuition is not treated as an independent business entity but as a value derived from validated learning activities. As a consequence, financial information remains consistent with attendance records and class execution results. If a class session is canceled or a student does not attend, that information automatically influences the calculated tuition amount without requiring additional manual adjustments.

One technical challenge arises from the fact that tuition generation may be executed multiple times throughout an academic month. Attendance records can be updated after class completion, teachers may correct previous attendance statuses and administrators may regenerate tuition data for reconciliation purposes. Consequently, the system must guarantee that repeated executions do not create duplicate tuition records or produce inconsistent financial results.

A noteworthy aspect of this solution is the tuition recalculation mechanism when source data changes. In practical operations, teachers may finalize attendance tracking after the initial tuition has been generated or update modifications to the status

of a past class session. If tuition data is fixed immediately after the first calculation, the outcome may no longer correctly reflect the actual academic situation.

To resolve this issue, EduCenter does not disregard existing tuition records. Upon receiving a tuition generation request, the system first checks the existence of the tuition record corresponding to the student, class and academic month. If the record does not exist yet, the system creates a new record with an unpaid status. Conversely, if the record already exists, the system performs a recalculation of the billable sessions and the total tuition amount, subsequently updating the existing record instead of generating an additional new one.

This behavior effectively implements an idempotent generation strategy. Regardless of how many times the tuition generation process is executed for the same student, class and month combination, the final result remains consistent because the system updates the existing record rather than creating duplicate entries. This characteristic is particularly important in educational management systems where source data may evolve after the initial calculation.

Based on the mechanism above, the monthly tuition generation algorithm of EduCenter can be described at an abstract level as shown in Pseudo-code 5.1. The algorithm uses only data that has been confirmed during the learning process, including approved enrollment information, completed class sessions and the attendance results of each student. When a tuition record for the same student, class and month already exists, the system updates the calculation results rather than creating a new record.

Pseudo-code 5.1. Monthly Tuition Generation Algorithm

FOR each approved enrollment in the selected month
  completedSessions ← get completed sessions of the class
  billableSessions ← count attendance records with PRESENT or LATE status
  totalAmount ← billableSessions × tuitionPerSession
  IF monthlyFee(student, class, month) exists
  THEN update totalSessions, unitPrice, totalAmount, dueDate
  ELSE create monthlyFee with UNPAID status
  END IF
END FOR

Table 5.1: Monthly Tuition Generation Algorithm

Pseudo-code 5.1 demonstrates that tuition in EduCenter is treated as derived data from actual learning activities rather than data entered directly by users. The algorithm performs data aggregation from multiple distinct sources, including en-

rollment information, class session statuses and attendance results, subsequently determining the number of billable sessions and the total amount to be collected automatically. A significant aspect is that the algorithm can be executed multiple times while maintaining data consistency. When source data changes, the system updates the existing tuition record instead of creating a new one. This approach ensures that tuition always properly reflects the actual academic status of the student and mitigates the risk of generating duplicate data.

5.2.3 Achieved Results

The proposed solution has automated the majority of the tuition calculation workflow at the center. Administrative personnel no longer need to manually cross-reference student lists, class schedules and attendance data to determine the amount to be collected for each student. The entire process of data aggregation and calculation is performed automatically based on existing information within the system. More importantly, the solution establishes a direct relationship between academic activities and financial records. Tuition amounts are generated from verified attendance information rather than from manually maintained financial data. This reduces discrepancies between operational and financial information while improving the traceability of tuition calculations.

Calculating tuition based on actual attendance data also increases overall system transparency. Students and parents can reconcile the tuition amount against the class sessions attended, thereby limiting disputes arising from discrepancies during the calculation process.

Unlike many tutoring center management systems that treat tuition merely as manually entered data or calculate it according to projected class sessions, EduCenter regards tuition as data derived from actual teaching and learning activities. The recalculation mechanism when source data changes ensures that the system always correctly reflects the academic status of students and maintains data consistency throughout the operational process. From a software engineering perspective, the solution demonstrates how financial information can be derived automatically from operational data while preserving consistency across multiple system modules. By combining attendance tracking, class management and tuition processing into a unified workflow, EduCenter reduces manual intervention and establishes a more reliable foundation for educational center operations.

5.3 Automated Schedule Generation and Teaching Conflict Detection Solution

5.3.1 Issue Introduction

Within tutoring center operations, constructing class schedules represents one of the most time-consuming workflows and is highly susceptible to human errors. Each class is typically organized following a repeating schedule spanning multiple weeks or months, resulting in a total volume that can reach dozens of individual sessions. If administrative personnel must manually create each discrete session, the operational workload increases significantly and the risk of omission or entering incorrect academic dates becomes unavoidable.

Furthermore, teacher assignment poses another distinct challenge for administrative operations. A single teacher may simultaneously take charge of multiple classes with different timetables. Without strict verification during the scheduling process, a teacher could be assigned to instruct two separate classes at the exact same time or within overlapping intervals. These scheduling conflicts not only complicate coordination efforts but also directly affect the operational quality of the center.

Many contemporary management systems still require users to create individual class sessions manually or independently cross-reference the teaching schedules of instructors prior to saving class profiles. This approach relies heavily on the experience of administrative personnel and fails to guarantee accuracy as the volume of classes continuously expands.

Driven by this reality, EduCenter aims to develop a mechanism that automatically generates class schedules from a minimal set of initial configuration parameters, while integrating a teaching schedule conflict detection mechanism directly into the class creation and update workflows. The solution reduces manual operations, safeguards data consistency and optimizes the deployment of the teaching resources of the center. An alternative approach would require administrators to manually create every individual class session and independently verify potential teacher conflicts before saving class information. While straightforward to implement, this approach becomes increasingly inefficient as the number of classes grows. The probability of missing sessions, assigning incorrect dates or overlooking teaching conflicts also increases significantly. EduCenter therefore adopts an automated scheduling strategy in which repetitive session generation and conflict verification are performed systematically by the system rather than relying on manual inspection.

5.3.2 Solution

A key design principle of the solution is the separation between schedule configuration data and generated session data. Instead of treating every class session as manually maintained information, EduCenter stores only the essential scheduling parameters of a class, including the start date, end date, teaching days and teaching hours. Individual sessions are subsequently derived from this configuration. This approach reduces data redundancy and ensures that all generated sessions remain consistent with the original class definition. Instead of requiring users to manually input each specific session, the system merely collects fundamental information about the class, encompassing the start date, end date, designated days of the week and the teaching timeframes. Leveraging these configuration parameters, the system automatically generates the comprehensive list of sessions for the class.

Figure 5.4 illustrates the automated schedule generation workflow within EduCenter. Upon receiving the configuration profile of a class, the system sequentially iterates through each calendar date between the start and end dates. For each specific date, the system validates whether the current day belongs to the configured list of academic days of the week. If this condition is satisfied, a new session record is generated with an initial scheduled status. This process continues until the entire timeframe of the class has been processed. The generation process follows a deterministic strategy. Given the same scheduling parameters, the algorithm always produces the same sequence of class sessions. This characteristic simplifies maintenance and ensures predictable system behavior when classes are modified or regenerated.

This approach yields two critical advantages. First, the administrator declares the class schedule only once rather than manually building each discrete session. Second, all sessions of a class are created following a uniform set of rule parameters, ensuring data homogeneity and limiting manual handling errors.

In addition to automated schedule generation, EduCenter incorporates a conflict detection mechanism for the teaching schedules of instructors. Figure 5.5 describes the conflict verification workflow within the system. When a manager creates or updates a class profile, the system first retrieves existing active classes assigned to the same teacher. Subsequently, the system executes two sequential verification checkpoints.

Rather than directly comparing teaching time intervals against every existing class, EduCenter performs conflict detection in two sequential stages. The objective is to eliminate impossible conflict scenarios as early as possible and reduce

unnecessary processing. The first checkpoint involves validating intersections between the sets of academic days. The system determines whether the new class and the existing classes of the teacher share at least one overlapping day of the week. If no common day exists, the new class is immediately recognized as conflict-free.

The second checkpoint is executed when two classes share at least one overlapping day of the week. The system proceeds to analyze intersections between the teaching timeframes by comparing the start and end times of both classes. Only when both conditions are simultaneously met-encompassing matching days and overlapping times-does the system designate the scenario as a teaching schedule conflict. This two-phase validation strategy improves efficiency because time interval comparisons are executed only when overlapping teaching days already exist. Classes scheduled on different days are discarded immediately without requiring further analysis. As the number of classes assigned to a teacher increases, this approach reduces the amount of unnecessary conflict-checking operations performed by the system.

The teaching schedule conflict detection algorithm of EduCenter can be described at an abstract level as shown in Pseudo-code 5.3.

Pseudo-code 5.3. Schedule Conflict Detection Algorithm

```
FOR each existingClass of the same teacher
   IF overlapDay(existingClass, newClass)
   AND overlapTime(existingClass, newClass)
   THEN reject class creation or update
END FOR
```

Table 5.2: Schedule Conflict Detection Algorithm

Pseudo-code 5.3 demonstrates that the conflict detection mechanism is executed by sequentially checking conditions from basic to more complex levels. Verifying intersections between sets of academic days prior to processing timeframes filters out cases where conflicts are mathematically impossible, thereby reducing unnecessary time-comparison operations. Upon detecting a conflict, the system blocks the data-saving process and returns detailed information regarding the overlapping classes, allowing the manager to easily adjust the teaching schedule accordingly.

5.3.3 Achieved Results

The proposed solution has automated the process of schedule construction within the EduCenter system. From a minimal set of initial configuration variables, the system can automatically generate the entire list of class sessions without requiring users to manually input each session individually. This minimizes the operational

workload of administrative personnel, particularly for long-term courses extending over multiple months. More importantly, the solution transforms schedule management from a data-entry activity into a rule-driven generation process. Administrative personnel only need to define a small set of scheduling parameters, while the system is responsible for constructing the complete sequence of learning sessions automatically.

The teaching schedule conflict detection mechanism also enhances the utilization efficiency of center teaching resources. The system proactively restricts instances where teachers are concurrently assigned to multiple classes with overlapping timetables, thereby minimizing coordination errors and reducing the risk of adjusting schedules after class profiles have been established.

Unlike many tutoring center management systems that solely support schedule information storage, EduCenter integrates automated mechanisms to assist in the organization and control of teaching schedules. This solution reduces manual work and improves data consistency, increasing system reliability and supporting more efficient center operations as the volume of classes and teachers continuously grows. From a software engineering perspective, the solution demonstrates how repetitive scheduling activities can be automated through configuration-driven generation and incremental conflict validation. Combining these mechanisms enables EduCenter to maintain consistent schedule data while reducing the operational complexity of class management.

This chapter has presented three major technical solutions implemented within the EduCenter system. The first solution transformed regulatory requirements into executable business rules and applied them through a multi-layer validation mechanism to support enrollment control. The second solution established a tuition generation process based on verified attendance data, ensuring consistency between academic activities and financial records. The third solution automated schedule construction and incorporated teaching conflict detection to reduce manual workload and improve schedule reliability. Beyond their individual functions, these solutions illustrate a common design philosophy adopted throughout the project: operational data should be validated, derived and controlled automatically whenever possible rather than relying solely on manual administrative actions. The resulting system not only digitizes tutoring center operations but also provides mechanisms that help maintain data consistency, regulatory compliance and operational efficiency. The outcomes discussed in this chapter form the basis for the overall evaluation and conclusions presented in the next chapter.

CHAPTER 6. CONCLUSION AND FUTURE WORK

6.1 Conclusion

Compared with many existing tutoring center management systems that primarily focus on digitizing administrative records and supporting routine management activities, EduCenter was designed to provide a higher level of operational automation and business-rule enforcement. The system integrates tutoring center management with regulatory compliance mechanisms, automated tuition generation and intelligent schedule management, enabling it to actively support operational decision-making rather than functioning solely as a digital information repository.

Within the scope of this thesis, the EduCenter tutoring center management system was successfully designed and implemented as a web application. The system supports the digitization of essential tutoring center operations, including user management, class management, student enrollment, teaching activity tracking, tuition management, notification delivery and statistical reporting. A layered architecture based on Next.js, Express.js and MySQL was adopted to separate presentation, business logic and data management, thereby improving maintainability and facilitating future system expansion.

During the execution of the thesis, the primary objectives defined for the project were successfully achieved within the intended scope. The system established a database capable of supporting tutoring center operations, implemented role-based access control for multiple user groups, developed responsive user interfaces for desktop and mobile devices and integrated the core workflows required for tutoring center management. Functional testing demonstrated that the implemented features operated correctly and satisfied the project requirements within the evaluation environment.

Alongside the achieved results, the system still possesses certain limitations. The deployment was newly conducted within a testing environment with a restricted number of users. Several advanced functionalities, such as online payment integration, real-time synchronization mechanisms, schedule recommendation systems, or learning data analytics tools, have not yet been implemented. In addition, the system has not yet been deployed in a production environment with long-term operational data or multiple tutoring centers. Consequently, its scalability, long-term reliability and performance under high user concurrency remain subjects for future evaluation.

The development of the EduCenter system also provided valuable practical lessons throughout the project. One important observation was that accurately modeling business rules and operational workflows required considerably more effort than implementing individual technical functionalities. Thorough requirement analysis during the early stages significantly reduced later design modifications and simplified system implementation. The project also demonstrated the importance of selecting an appropriate software architecture to maintain clear responsibility separation, facilitate testing and support future system evolution.

6.2 Future Work

In the coming period, the system can be continuously refined across multiple directions to enhance its practical feasibility and applicability within real-world operational environments.

First and foremost, several existing functionalities require continuous optimization. The system could incorporate a mechanism to automatically regenerate class schedules whenever class information changes, expand the tuition management workflow to support multiple payment statuses and integrate online payment gateways. Conducting practical testing and deployment with a larger user base is also necessary to evaluate the load capacity and stability of the system.

In addition to refining current functionalities, the system can be expanded into several new directions. A potential development path involves building mobile applications for students and teachers to increase accessibility and support multi-platform utilization. The system could also integrate real-time notification mechanisms, synchronize class schedules with popular calendar services and construct analytical dashboards to serve center administration.

Looking further ahead, EduCenter could be extended through several advanced research and development directions. One promising direction is the application of data analytics and artificial intelligence techniques to forecast enrollment demand, recommend appropriate class schedules and provide analytical insights for tutoring center administrators. Another potential enhancement involves extending the system to support multi-branch tutoring centers, enabling centralized management of teachers, students and classes across different locations. These future developments would further improve the practicality of the system and strengthen its ability to support educational management in larger and more complex operational environments.

REFERENCE

[1] Ministry of Education and Training of Vietnam, *Circular no. 29/2024/tt-bgdđt on regulations for extra teaching and learning*, https://moet.gov.vn, Issued on December 30, 2024, 2024. Accessed: Jun. 28, 2026.

[2] Mona Media, *Mona elms - education management system*, https://mona.media, 2025. Accessed: Jun. 28, 2026.

[3] Faceworks, *Faceworks education management*, https://faceworks.vn, 2025. Accessed: Jun. 28, 2026.

[4] EasyEdu, *Easyedu education management software*, https://easyedu.vn, 2025. Accessed: Jun. 28, 2026.

[5] Microsoft, *Typescript documentation*, https://www.typescriptlang.org/docs/, 2025. Accessed: Jun. 28, 2026.

[6] Meta, *React documentation*, https://react.dev/, 2025. Accessed: Jun. 28, 2026.

[7] Vercel, *Next.js documentation*, https://nextjs.org/docs, 2025. Accessed: Jun. 28, 2026.

[8] Tailwind Labs, *Tailwind css documentation*, https://tailwindcss.com/docs, 2025. Accessed: Jun. 28, 2026.

[9] PMNDRS, *Zustand documentation*, https://zustand.docs.pmnd.rs/, 2025. Accessed: Jun. 28, 2026.

[10] OpenJS Foundation, *Node.js documentation*, https://nodejs.org/docs/latest/api/, 2025. Accessed: Jun. 28, 2026.

[11] Express.js, *Express.js documentation*, https://expressjs.com/, 2025. Accessed: Jun. 28, 2026.

[12] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[13] M. Jones, J. Bradley, and N. Sakimura, *Json web token (jwt)*, RFC 7519, 2015.

[14] D. F. Ferraiolo and D. R. Kuhn, "Role-based access controls," *15th National Computer Security Conference*, pp. 554–563, 1992.

[15] Oracle, *Mysql 8.0 reference manual*, https://dev.mysql.com/doc/, 2025. Accessed: Jun. 28, 2026.

APPENDIX